Lecture 8: Introduction to Unsupervised Learning, K-means, Gaussian Mixture Model (GMM), and EM algorithm

Tao LIN

SoE, Westlake University

October 27, 2026

WESTLAKE UNIVERSITY | SCHOOL OF ENGINEERING

1. Complete the generative classification story
   - GDA: overview and maximum likelihood
   - Classifying emails: build a model from the data

2. Unsupervised Learning
   - Introduction to Unsupervised Learning
   - Clustering and K-means Methods
   - Gaussian Mixture Model (GMM)
   - Expectation-Maximization Algorithm

Reading materials & Reference

Reading materials:

- Chapter 10, Stanford CS 229 Lecture Notes, `https://cs229.stanford.edu/notes2022fall/main_notes.pdf`
- EPFL, CS-433 Machine Learning, `https://github.com/epfml/ML_course`

Table of Contents

1 Complete the generative classification story
- GDA: overview and maximum likelihood
- Classifying emails: build a model from the data

2 Unsupervised Learning

Table of Contents

1. Complete the generative classification story
   - GDA: overview and maximum likelihood
   - Classifying emails: build a model from the data
2. Unsupervised Learning
   - Introduction to Unsupervised Learning
   - Clustering and K-means Methods
   - Gaussian Mixture Model (GMM)
   - Expectation-Maximization Algorithm

GDA overview: model, fit, then classify

Logistic regression fits $p(y \mid \mathbf{x})$ directly. GDA first models the inputs within each class:

1. **Model each class:** $p(y = c) = \pi_c, \quad p(\mathbf{x} \mid y = c) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.

GDA overview: model, fit, then classify

Logistic regression fits $p(y \mid \mathbf{x})$ directly. GDA first models the inputs within each class:

1. **Model each class:** $p(y = c) = \pi_c, \quad p(\mathbf{x} \mid y = c) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.
2. **Train using known labels:** estimate class proportions, centers, and spreads.

GDA overview: model, fit, then classify

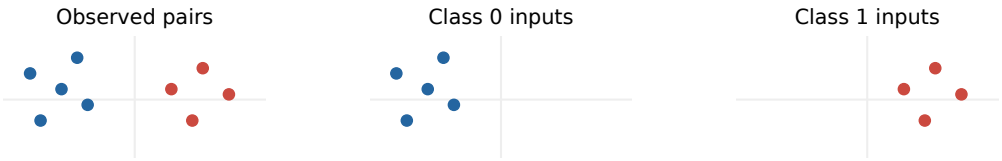
Logistic regression fits $p(y \mid \mathbf{x})$ directly. GDA first models the inputs within each class:

1. **Model each class:** $p(y = c) = \pi_c, \quad p(\mathbf{x} \mid y = c) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.
2. **Train using known labels:** estimate class proportions, centers, and spreads.
3. **Predict for a new input:** compute one weight per class,

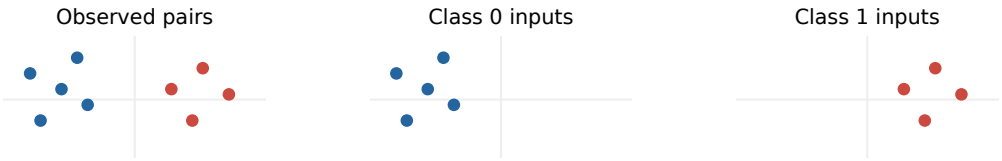
$$w_c(\mathbf{x}) = \hat{\pi}_c \mathcal{N}(\mathbf{x} \mid \hat{\boldsymbol{\mu}}_c, \hat{\boldsymbol{\Sigma}}_c), \qquad p(y = c \mid \mathbf{x}) = \frac{w_c(\mathbf{x})}{\sum_k w_k(\mathbf{x})}.$$

Choose the class with the largest weight.

GDA training: the labels tell us which Gaussian to fit



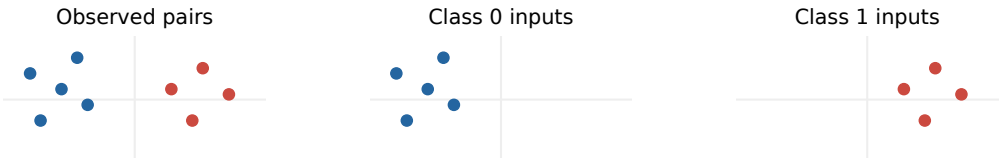
GDA training: the labels tell us which Gaussian to fit



Choose parameters that make the observed labeled data likely:

$$L(\theta) = \prod_n \pi_{y_n} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_{y_n}, \boldsymbol{\Sigma}_{y_n}).$$

GDA training: the labels tell us which Gaussian to fit



Choose parameters that make the observed labeled data likely:

$$L(\theta) = \prod_n \pi_{y_n} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_{y_n}, \boldsymbol{\Sigma}_{y_n}).$$

Taking logs separates class frequencies from within-class fits:

$$\log L = \sum_c N_c \log \pi_c + sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c).$$

N_c counts training inputs with label c. Each label selects its Gaussian.

GDA MLE: count the class, then average its inputs

Class proportion: the fitted class probability matches its observed fraction.

$$\hat{\pi}_c = \frac{N_c}{N}.$$

GDA MLE: count the class, then average its inputs

Class proportion: the fitted class probability matches its observed fraction.

$$\hat{\pi}_c = \frac{N_c}{N}.$$

Class center: average only inputs whose label is c.

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

GDA MLE: count the class, then average its inputs

Class proportion: the fitted class probability matches its observed fraction.

$$\hat{\pi}_c = \frac{N_c}{N}.$$

Class center: average only inputs whose label is c.

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n.$$

For the mean, the likelihood stationary equation is

$$\sum_{n:y_n=c} \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_c) = 0 \quad \Longrightarrow \quad N_c \boldsymbol{\mu}_c = \sum_{n:y_n=c} \mathbf{x}_n.$$

Every input contributes fully to its known class.

GDA MLE: measure spread around each class mean

First subtract the fitted mean of the input's own class:

$$\mathbf{r}_n = \mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n}.$$

GDA MLE: measure spread around each class mean

First subtract the fitted mean of the input's own class:

$$\mathbf{r}_n = \mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n}.$$

QDA: a separate covariance for each class.

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{r}_n \mathbf{r}_n^\top.$$

GDA MLE: measure spread around each class mean

First subtract the fitted mean of the input's own class:

$$\mathbf{r}_n = \mathbf{x}_n - \hat{\boldsymbol{\mu}}_{y_n}.$$

QDA: a separate covariance for each class.

$$\hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{r}_n \mathbf{r}_n^\top.$$

LDA: one covariance shared by all classes.

$$\hat{\boldsymbol{\Sigma}} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{r}_n \mathbf{r}_n^\top.$$

Pool the within-class residuals, rather than subtracting the overall mean.

GDA by hand: fit first, then classify a new input

	Class 0	Class 1
Labeled training inputs	$0,\ 2$	$4,\ 6$

GDA by hand: fit first, then classify a new input

	Class 0	Class 1
Labeled training inputs	$0,\ 2$	$4,\ 6$
Fitted prior	$1/2$	$1/2$
Fitted mean	1	5

GDA by hand: fit first, then classify a new input

	Class 0	Class 1
Labeled training inputs	$0,\ 2$	$4,\ 6$
Fitted prior	$1/2$	$1/2$
Fitted mean	1	5
Fitted variance	$[(0-1)^2 + (2-1)^2]/2 = 1$	1

The two variances agree, so the shared LDA variance is also 1.

GDA by hand: fit first, then classify a new input

	Class 0	Class 1
Labeled training inputs	$0,\ 2$	$4,\ 6$
Fitted prior	$1/2$	$1/2$
Fitted mean	1	5
Fitted variance	$[(0-1)^2+(2-1)^2]/2=1$	1

The two variances agree, so the shared LDA variance is also 1.

For a new input $x=4$, compare the two fitted class weights:

$$\frac{w_1}{w_0}=\frac{\frac{1}{2}\mathcal{N}(4\mid 5,1)}{\frac{1}{2}\mathcal{N}(4\mid 1,1)}=e^{(9-1)/2}=e^4, \qquad p(y=1\mid x=4)=\frac{e^4}{1+e^4}\approx 0.982.$$

Predict class 1. Training used labels; prediction uses the fitted distributions.

Table of Contents

1. Complete the generative classification story
   - GDA: overview and maximum likelihood
   - Classifying emails: build a model from the data

2. Unsupervised Learning
   - Introduction to Unsupervised Learning
   - Clustering and K-means Methods
   - Gaussian Mixture Model (GMM)
   - Expectation-Maximization Algorithm

A new task: classify emails

GDA used a Gaussian to describe continuous inputs within each class.

A new task: classify emails

GDA used a Gaussian to describe continuous inputs within each class.

Now consider an email: “`special offer today`”.

Our classes are spam and non-spam, and our inputs are words.

A new task: classify emails

GDA used a Gaussian to describe continuous inputs within each class.

Now consider an email: "`special offer today`".

Our classes are spam and non-spam, and our inputs are words.

The classification question is familiar: which class better explains this input?

First, how should we represent an email?

Start with the data: which words occur?

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Start with the data: which words occur?

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

For dictionary `[offer, meeting]`,

"`offer offer`" becomes $\mathbf{x} = (1, 0)$.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Start with the data: which words occur?

Task: classify an email as spam ($y = 1$) or non-spam ($y = 0$).
Fix a dictionary of words. For each word j,

$$x_j = \begin{cases} 1, & \text{the word occurs in the email,} \\ 0, & \text{the word does not occur.} \end{cases}$$

For dictionary `[offer, meeting]`,

"`offer offer`" becomes $\mathbf{x} = (1, 0)$.

We record **presence**, not the number of occurrences.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Could we count complete email patterns?

For dictionary `[offer, meeting]`, there are four possible inputs:

`offer`	`meeting`	Pattern
0	0	Neither word
1	0	Only `offer`
0	1	Only `meeting`
1	1	Both words

Could we count complete email patterns?

For dictionary `[offer, meeting]`, there are four possible inputs:

`offer`	`meeting`	Pattern
0	0	Neither word
1	0	Only `offer`
0	1	Only `meeting`
1	1	Both words

We could count each complete pattern within each class.

But d binary words give 2^d patterns; many may have few or no examples.

Could we count complete email patterns?

For dictionary `[offer, meeting]`, there are four possible inputs:

`offer`	`meeting`	Pattern
0	0	Neither word
1	0	Only `offer`
0	1	Only `meeting`
1	1	Both words

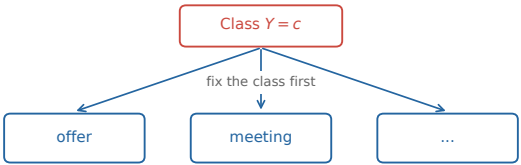
We could count each complete pattern within each class.

But d binary words give 2^d patterns; many may have few or no examples.

Can we build the email probability from simpler, per-word probabilities?

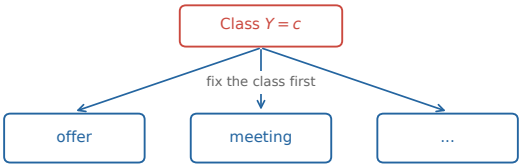
A simplifying assumption: model words within each class

For spam, estimate each word's presence probability separately; do the same for non-spam.



A simplifying assumption: model words within each class

For spam, estimate each word's presence probability separately; do the same for non-spam.

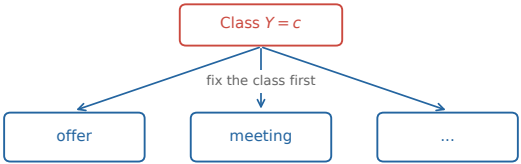


Assume that, **once the class is fixed**, knowing other words does not change a word's probability:

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^{d} p(x_j \mid y = c).$$

A simplifying assumption: model words within each class

For spam, estimate each word's presence probability separately; do the same for non-spam.



Assume that, **once the class is fixed**, knowing other words does not change a word's probability:

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^{d} p(x_j \mid y = c).$$

This gives **Bernoulli Naïve Bayes**. "Naïve" refers to the conditional-independence assumption; real words can still be related.

What probabilities do we need?

For each candidate class c, learn:

- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.

What probabilities do we need?

For each candidate class c, learn:

- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.
- **Word probability:** $q_{jc} = p(x_j = 1 \mid y = c)$,
  how often word j occurs in emails of this class.

What probabilities do we need?

For each candidate class c, learn:

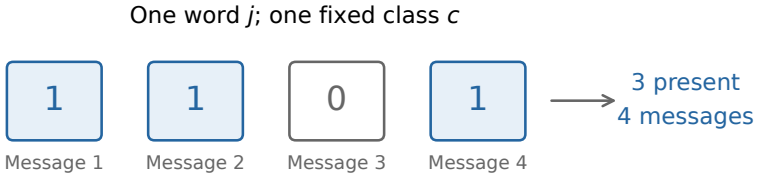
- **Class prior:** $\pi_c = p(y = c)$, the fraction of emails in this class.
- **Word probability:** $q_{jc} = p(x_j = 1 \mid y = c)$,
  how often word j occurs in emails of this class.

For a new email, each word selects one factor:

$$p(x_j \mid y = c) = \begin{cases} q_{jc}, & x_j = 1 \quad \text{(present)}, \\ 1 - q_{jc}, & x_j = 0 \quad \text{(absent)}. \end{cases}$$

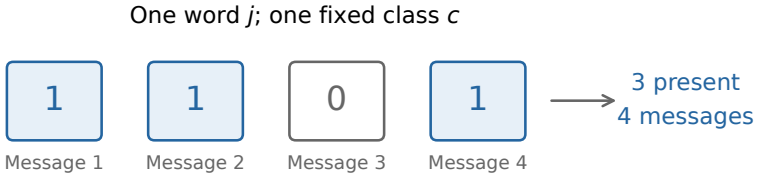
Training: count within each class

Suppose word j occurs in three of four emails in class c.



Training: count within each class

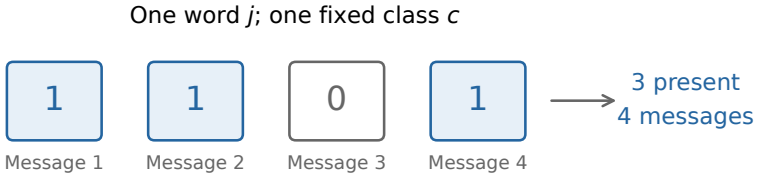
Suppose word j occurs in three of four emails in class c.



$$\hat{q}_{jc} = \frac{\text{class-}c\text{ emails containing word } j}{\text{all class-}c\text{ emails}} = \frac{N_{jc}}{N_c} = \frac{3}{4}.$$

Training: count within each class

Suppose word j occurs in three of four emails in class c.



$$\hat{q}_{jc} = \frac{\text{class-}c\text{ emails containing word } j}{\text{all class-}c\text{ emails}} = \frac{N_{jc}}{N_c} = \frac{3}{4}.$$

Repeat for every word and class. Estimate the prior by $\hat{\pi}_c = N_c/N$.

Training is just counting; no iterative optimizer is needed.

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} .$$

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} \quad .$$

Choose the class with the larger weight. Both present and absent words contribute.

Prediction: multiply, then compare

Use the learned probabilities to compute one weight for each class:

$$w_c = \underbrace{\pi_c}_{\text{class prior}} \prod_{j=1}^{d} \underbrace{p(x_j \mid y = c)}_{\text{present: } q_{jc} \text{ ; absent: } 1-q_{jc}} .$$

Choose the class with the larger weight. Both present and absent words contribute.

If we also want a probability, normalize the weights:

$$p(y = 1 \mid \mathbf{x}) = \frac{w_1}{w_0 + w_1}.$$

The weights already include the class priors: this is Bayes' rule.

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior	`offer`	no `meeting`	Weight

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior		`offer`		no `meeting`		Weight
Spam	0.5	$\times$	0.8	$\times$	$1 - 0.2$	$=$	0.32
Non-spam	0.5	$\times$	0.1	$\times$	$1 - 0.7$	$=$	0.015

One email, two words: work it out

Dictionary: `[offer, meeting]`. The email has `offer`, but no `meeting`: $\mathbf{x} = (1, 0)$.

	Prior		`offer`		no `meeting`		Weight
Spam	0.5	$\times$	0.8	$\times$	$1 - 0.2$	$=$	0.32
Non-spam	0.5	$\times$	0.1	$\times$	$1 - 0.7$	$=$	0.015

$$p(\text{spam} \mid \mathbf{x}) = \frac{0.32}{0.32 + 0.015} \approx 0.955.$$

Predict **spam**. For the absent word, use $1 - q$, not q.

Why smooth? A zero count becomes a zero probability

Suppose `offer` is absent from all three training emails labeled spam.

Training spam email	1	2	3
Contains `offer`?	0	0	0

Why smooth? A zero count becomes a zero probability

Suppose `offer` is absent from all three training emails labeled spam.

Training spam email	1	2	3
Contains `offer`?	0	0	0

Counting gives $\hat{p}(\texttt{offer present} \mid \text{spam}) = 0/3 = 0$.

For a new email that *does* contain `offer`,

$$w_{\text{spam}} = \pi_{\text{spam}} \times \underbrace{0}_{\texttt{offer}} \times \text{other word factors} = 0.$$

Why smooth? A zero count becomes a zero probability

Suppose `offer` is absent from all three training emails labeled spam.

Training spam email	1	2	3
Contains `offer`?	0	0	0

Counting gives $\hat{p}(\texttt{offer present} \mid \text{spam}) = 0/3 = 0$.

For a new email that *does* contain `offer`,

$$w_{\text{spam}} = \pi_{\text{spam}} \times \underbrace{0}_{\texttt{offer}} \times \text{other word factors} = 0.$$

Not observed in three emails does not mean impossible in future spam.

We need an estimate that leaves some probability for an event we have not seen.

Laplace smoothing: add one count to each outcome

Keep the same three spam emails. Add one extra count for each possible outcome:

`offer`	Observed counts
Present	0
Absent	3
Total	3

Laplace smoothing: add one count to each outcome

Keep the same three spam emails. Add one extra count for each possible outcome:

`offer`	Observed counts	Counts used for estimation
Present	0	$0 + 1 = 1$
Absent	3	$3 + 1 = 4$
Total	3	$3 + 2 = 5$

Laplace smoothing: add one count to each outcome

Keep the same three spam emails. Add one extra count for each possible outcome:

`offer`	Observed counts	Counts used for estimation
Present	0	$0 + 1 = 1$
Absent	3	$3 + 1 = 4$
Total	3	$3 + 2 = 5$

$$p(\texttt{present} \mid \textsf{spam}) = \frac{1}{5}, \qquad p(\texttt{absent} \mid \textsf{spam}) = \frac{4}{5}.$$

The two probabilities still sum to one, and neither is zero.

Laplace smoothing: add one count to each outcome

Keep the same three spam emails. Add one extra count for each possible outcome:

`offer`	Observed counts	Counts used for estimation
Present	0	$0 + 1 = 1$
Absent	3	$3 + 1 = 4$
Total	3	$3 + 2 = 5$

$$p(\texttt{present} \mid \text{spam}) = \frac{1}{5}, \qquad p(\texttt{absent} \mid \text{spam}) = \frac{4}{5}.$$

The two probabilities still sum to one, and neither is zero.

The general rule: $\tilde{q}_{jc} = \dfrac{N_{jc} + 1}{N_c + 2}$.

The denominator gains 2 because we added one count to *each of two outcomes*.

Naïve Bayes: the basic idea

- **Assume:** features are independent given the class.
- **Train:** count class frequencies and word presence within each class; add pseudo-counts to avoid zero word probabilities.

Naïve Bayes: the basic idea

- **Assume:** features are independent given the class.
- **Train:** count class frequencies and word presence within each class; add pseudo-counts to avoid zero word probabilities.
- **Predict:** multiply the prior by the word factors for each class, then choose the larger weight. Normalize if a probability is needed.

The same generative recipe as GDA: model $p(\mathbf{x} \mid y)$ and $p(y)$, then use Bayes' rule.

Overall recap: one generative classification recipe

	GDA	Bernoulli Naïve Bayes
Input model within a class	Multivariate Gaussian	Independent Bernoulli features
Training statistics	Counts, centers, spreads	Class and word counts

Overall recap: one generative classification recipe

	GDA	Bernoulli Naïve Bayes
Input model within a class	Multivariate Gaussian	Independent Bernoulli features
Training statistics	Counts, centers, spreads	Class and word counts

Both train with known class labels, then classify a new input using

$$w_c(\mathbf{x}) = p(y = c)\, p(\mathbf{x} \mid y = c), \qquad p(y = c \mid \mathbf{x}) = \frac{w_c(\mathbf{x})}{\sum_k w_k(\mathbf{x})}.$$

Overall recap: one generative classification recipe

	GDA	Bernoulli Naïve Bayes
Input model within a class	Multivariate Gaussian	Independent Bernoulli features
Training statistics	Counts, centers, spreads	Class and word counts

Both train with known class labels, then classify a new input using

$$w_c(\mathbf{x}) = p(y = c)\, p(\mathbf{x} \mid y = c), \qquad p(y = c \mid \mathbf{x}) = \frac{w_c(\mathbf{x})}{\sum_k w_k(\mathbf{x})}.$$

The labels made training easy: they told us which inputs to group together.

What changes if we erase the training labels?

Table of Contents

1 Complete the generative classification story

2 Unsupervised Learning
- Introduction to Unsupervised Learning
- Clustering and K-means Methods
- Gaussian Mixture Model (GMM)
- Expectation-Maximization Algorithm

Table of Contents

1 Complete the generative classification story
- GDA: overview and maximum likelihood
- Classifying emails: build a model from the data

2 Unsupervised Learning
- Introduction to Unsupervised Learning
- Clustering and K-means Methods
- Gaussian Mixture Model (GMM)
- Expectation-Maximization Algorithm

From labeled classes to unlabeled groups

We just fitted class models using known training labels.

$$\{(\mathbf{x}_n, y_n)\}_{n=1}^N \quad \longrightarrow \quad \{\mathbf{x}_n\}_{n=1}^N$$

From labeled classes to unlabeled groups

We just fitted class models using known training labels.

$$\{(\mathbf{x}_n, y_n)\}_{n=1}^N \quad \longrightarrow \quad \{\mathbf{x}_n\}_{n=1}^N$$

Today, the inputs remain, but the group labels are hidden.

From labeled classes to unlabeled groups

We just fitted class models using known training labels.

$$\{(\mathbf{x}_n, y_n)\}_{n=1}^N \qquad \longrightarrow \qquad \{\mathbf{x}_n\}_{n=1}^N$$

Today, the inputs remain, but the group labels are hidden.

Our route: choose centers $\rightarrow$ assign points $\rightarrow$ refit centers.

From labeled classes to unlabeled groups

We just fitted class models using known training labels.

$$\{(\mathbf{x}_n, y_n)\}_{n=1}^N \qquad \longrightarrow \qquad \{\mathbf{x}_n\}_{n=1}^N$$

Today, the inputs remain, but the group labels are hidden.

Our route: choose centers $\rightarrow$ assign points $\rightarrow$ refit centers.

Then replace definite assignments by probabilities, and learn a density.

What can we learn without labels?

We observe $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^D$; no target y_n is supplied.

What can we learn without labels?

We observe $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^D$; no target y_n is supplied.

- **Representation:** which features summarize the inputs?

What can we learn without labels?

We observe $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^D$; no target y_n is supplied.

- **Representation:** which features summarize the inputs?
- **Clustering:** which inputs are similar under our chosen metric?

What can we learn without labels?

We observe $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^D$; no target y_n is supplied.

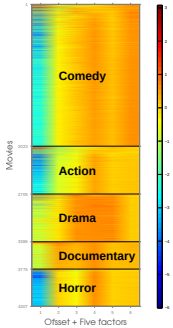
- **Representation:** which features summarize the inputs?
- **Clustering:** which inputs are similar under our chosen metric?
- **Density estimation:** which inputs are plausible under a model?

This lecture develops clustering and density estimation using K-means and GMMs.

More representation and generation examples (optional).

Example

Given ratings of movies and viewers, we use matrix factorization to extract useful features (see e.g. Netflix Prize).



Clustering more than two million biomedical publications (Kevin Boyack et.al. 2011)

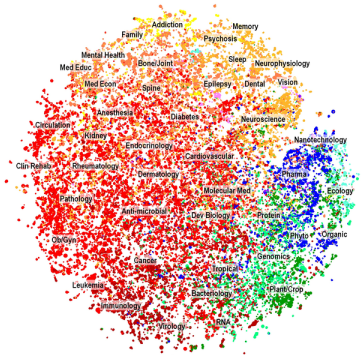


Table of Contents

1 Complete the generative classification story
- GDA: overview and maximum likelihood
- Classifying emails: build a model from the data

2 Unsupervised Learning
- Introduction to Unsupervised Learning
- Clustering and K-means Methods
- Gaussian Mixture Model (GMM)
- Expectation-Maximization Algorithm

Clustering: choose representatives and assignments

Fix the number of groups K and a distance between inputs.

Clustering: choose representatives and assignments

Fix the number of groups K and a distance between inputs.

- $\boldsymbol{\mu}_k \in \mathbb{R}^D$: center of group k.

Clustering: choose representatives and assignments

Fix the number of groups K and a distance between inputs.

- $\boldsymbol{\mu}_k \in \mathbb{R}^D$: center of group k.
- $z_n \in \{1, \ldots, K\}$: group chosen for input $\mathbf{x}_n$.

Clustering: choose representatives and assignments

Fix the number of groups K and a distance between inputs.

- $\boldsymbol{\mu}_k \in \mathbb{R}^D$: center of group k.
- $z_n \in \{1, \ldots, K\}$: group chosen for input $\mathbf{x}_n$.

Question: can we make each input close to its chosen center?

$$J = \sum_{n=1}^{N} \|\mathbf{x}_n - \boldsymbol{\mu}_{z_n}\|_2^2$$

K-means clustering: read one term first

Write the assignment as an indicator: $z_{nk} = 1$ if $z_n = k$, otherwise 0.

K-means clustering: read one term first

Write the assignment as an indicator: $z_{nk} = 1$ if $z_n = k$, otherwise 0.

$$\mathbf{z}_n = (0, 1, 0)^\top \quad \Longrightarrow \quad \sum_{k=1}^{3} z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 = \|\mathbf{x}_n - \boldsymbol{\mu}_2\|^2.$$

K-means clustering: read one term first

Write the assignment as an indicator: $z_{nk} = 1$ if $z_n = k$, otherwise 0.

$$\mathbf{z}_n = (0, 1, 0)^\top \quad \Longrightarrow \quad \sum_{k=1}^{3} z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 = \|\mathbf{x}_n - \boldsymbol{\mu}_2\|^2.$$

Now add the contributions of all inputs:

$$\min_{\mathbf{z}, \boldsymbol{\mu}} J(\mathbf{z}, \boldsymbol{\mu}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2^2$$

K-means clustering: read one term first

Write the assignment as an indicator: $z_{nk} = 1$ if $z_n = k$, otherwise 0.

$$\mathbf{z}_n = (0, 1, 0)^\top \quad \Longrightarrow \quad \sum_{k=1}^{3} z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 = \|\mathbf{x}_n - \boldsymbol{\mu}_2\|^2.$$

Now add the contributions of all inputs:

$$\min_{\mathbf{z}, \boldsymbol{\mu}} J(\mathbf{z}, \boldsymbol{\mu}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2^2$$

$z_{nk} \in \{0, 1\}$ and $\sum_k z_{nk} = 1$: exactly one center per input.

Both the assignments and the centers are unknown. Which can we fix first?

Bridge: one K-means iteration by hand

Use $x = (0, 2, 4)$, $K = 2$, and initial centers $(\mu_1, \mu_2) = (0, 4)$.

Input x_n	0	2	4
Squared distance to 0	0	4	16
Squared distance to 4	16	4	0

Bridge: one K-means iteration by hand

Use $x = (0, 2, 4)$, $K = 2$, and initial centers $(\mu_1, \mu_2) = (0, 4)$.

Input x_n	0	2	4
Squared distance to 0	0	4	16
Squared distance to 4	16	4	0
Chosen group (tie $\to 1$)	1	1	2

Bridge: one K-means iteration by hand

Use $x = (0, 2, 4)$, $K = 2$, and initial centers $(\mu_1, \mu_2) = (0, 4)$.

Input x_n	0	2	4
Squared distance to 0	0	4	16
Squared distance to 4	16	4	0
Chosen group (tie $\to 1$)	1	1	2

New centers: $\mu_1 = (0 + 2)/2 = 1$, $\mu_2 = 4$.

Bridge: one K-means iteration by hand

Use $x = (0, 2, 4)$, $K = 2$, and initial centers $(\mu_1, \mu_2) = (0, 4)$.

Input x_n	0	2	4
Squared distance to 0	0	4	16
Squared distance to 4	16	4	0
Chosen group (tie $\to 1$)	1	1	2

New centers: $\mu_1 = (0 + 2)/2 = 1$, $\mu_2 = 4$.

Old cost $J = 4$; new cost $J = 1 + 1 + 0 = 2$.
We will reuse these three inputs when assignments become soft.

K-means Methods: solve one unknown at a time

Choose initial centers $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$.

K-means Methods: solve one unknown at a time

Choose initial centers $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$.

1. **Freeze centers.** Assign each input to its nearest center.

K-means Methods: solve one unknown at a time

Choose initial centers $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$.

1. **Freeze centers.** Assign each input to its nearest center.
2. **Freeze assignments.** Move each center to its group's mean.

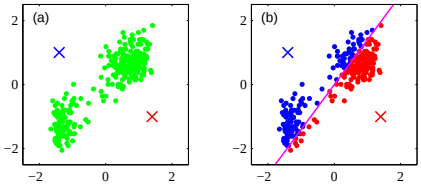
K-means Methods: solve one unknown at a time

Choose initial centers $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$.

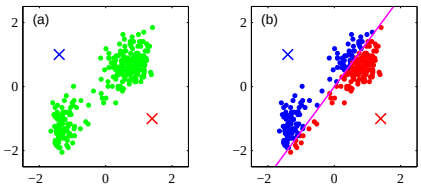
1. **Freeze centers.** Assign each input to its nearest center.
2. **Freeze assignments.** Move each center to its group's mean.

Repeat. Each step solves an easier problem with the other unknown fixed.

Step 1: freeze centers, choose the nearest

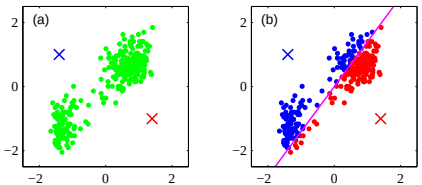


Step 1: freeze centers, choose the nearest



For one input, compare $\|\mathbf{x}_n - \boldsymbol{\mu}_1\|^2, \ldots, \|\mathbf{x}_n - \boldsymbol{\mu}_K\|^2$.

Step 1: freeze centers, choose the nearest

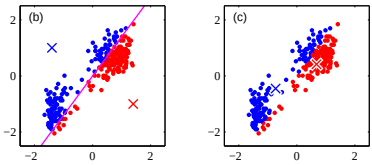


For one input, compare $\|\mathbf{x}_n - \boldsymbol{\mu}_1\|^2, \ldots, \|\mathbf{x}_n - \boldsymbol{\mu}_K\|^2$.

$$z_n = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2, \qquad z_{nk} = \mathbf{1}\{z_n = k\}.$$

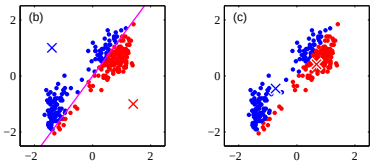
Each input can be assigned independently. Break ties by a fixed rule.

Step 2: freeze assignments, move to the mean



Only cluster k's terms depend on $\boldsymbol{\mu}_k$:

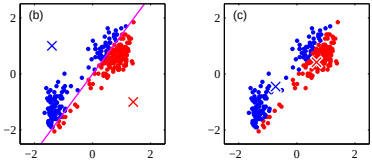
Step 2: freeze assignments, move to the mean



Only cluster k's terms depend on $\boldsymbol{\mu}_k$:

$$\nabla_{\boldsymbol{\mu}_k} J = 2 \sum_n z_{nk} (\boldsymbol{\mu}_k - \mathbf{x}_n) = 0$$

Step 2: freeze assignments, move to the mean



Only cluster k's terms depend on $\boldsymbol{\mu}_k$:

$$\nabla_{\boldsymbol{\mu}_k} J = 2 \sum_n z_{nk} (\boldsymbol{\mu}_k - \mathbf{x}_n) = 0$$

$$\boldsymbol{\mu}_k = \frac{\sum_n z_{nk} \mathbf{x}_n}{\sum_n z_{nk}} \quad \text{if } \sum_n z_{nk} > 0.$$

Sum of assigned inputs ÷ number of assigned inputs.

Summary of K-means

1. Initialize K centers.

Summary of K-means

1. Initialize K centers.
2. Assign: $z_n^{(t+1)} = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j^{(t)}\|^2$.

Summary of K-means

1. Initialize K centers.
2. Assign: $z_n^{(t+1)} = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j^{(t)}\|^2$.
3. Refit: $\boldsymbol{\mu}_k^{(t+1)} = \dfrac{\sum_n z_{nk}^{(t+1)} \mathbf{x}_n}{\sum_n z_{nk}^{(t+1)}}$.

Summary of K-means

1. Initialize K centers.
2. Assign: $z_n^{(t+1)} = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j^{(t)}\|^2$.
3. Refit: $\boldsymbol{\mu}_k^{(t+1)} = \dfrac{\sum_n z_{nk}^{(t+1)} \mathbf{x}_n}{\sum_n z_{nk}^{(t+1)}}$.
4. Repeat until assignments stop changing or improvement is small.

Use a fixed tie rule; handle empty clusters; compare several initializations.

Coordinate descent: why the cost cannot increase

First optimize assignments with the old centers:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq J(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

Coordinate descent: why the cost cannot increase

First optimize assignments with the old centers:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq J(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

Then optimize centers with the new assignments:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}) \leq J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}).$$

Coordinate descent: why the cost cannot increase

First optimize assignments with the old centers:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq J(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

Then optimize centers with the new assignments:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}) \leq J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}).$$

Together: J never increases. Since $J \geq 0$, its values converge.

Coordinate descent: why the cost cannot increase

First optimize assignments with the old centers:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq J(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

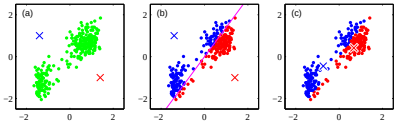
Then optimize centers with the new assignments:

$$J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}) \leq J(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}).$$

Together: J never increases. Since $J \geq 0$, its values converge.

Different initial centers can still lead to different final costs.
Neither step searches all assignments and centers jointly.

K-means for the “old-faithful” dataset (Bishop’s Figure 9.1)

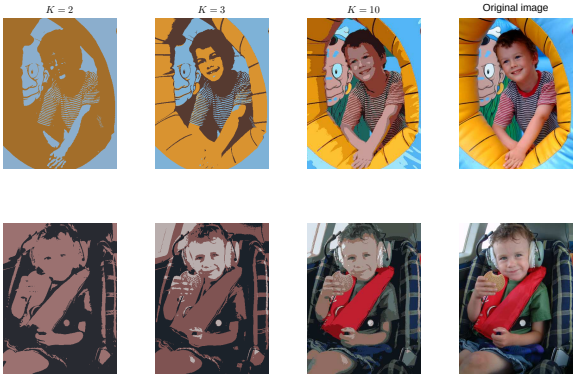


(a) Iteration 0 (b) Iteration 1 (c) Iteration 1

(d) Iteration 2 (e) Iteration 2 (f) Iteration 3

(g) Iteration 3 (h) Iteration 4 (i) Iteration 4

Data compression for images (this is also known as vector quantization).



K-means as reconstruction

Replace each input by its chosen center: $\widehat{\mathbf{x}}_n = \boldsymbol{\mu}_{z_n}$.

K-means as reconstruction

Replace each input by its chosen center: $\widehat{\mathbf{x}}_n = \boldsymbol{\mu}_{z_n}$.

$$J = \sum_n \|\mathbf{x}_n - \widehat{\mathbf{x}}_n\|^2.$$

K-means as reconstruction

Replace each input by its chosen center: $\widehat{\mathbf{x}}_n = \boldsymbol{\mu}_{z_n}$.

$$J = \sum_n \|\mathbf{x}_n - \widehat{\mathbf{x}}_n\|^2.$$

Stack the inputs and assignments by rows:

$$\mathbf{X} \in \mathbb{R}^{N \times D}, \quad \mathbf{Z} \in \{0,1\}^{N \times K}, \quad \mathbf{M} = [\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K] \in \mathbb{R}^{D \times K}.$$

K-means as reconstruction

Replace each input by its chosen center: $\widehat{\mathbf{x}}_n = \boldsymbol{\mu}_{z_n}$.

$$J = \sum_n \|\mathbf{x}_n - \widehat{\mathbf{x}}_n\|^2.$$

Stack the inputs and assignments by rows:

$$\mathbf{X} \in \mathbb{R}^{N \times D}, \quad \mathbf{Z} \in \{0,1\}^{N \times K}, \quad \mathbf{M} = [\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K] \in \mathbb{R}^{D \times K}.$$

$$J = \|\mathbf{X} - \mathbf{Z}\mathbf{M}^\top\|_{\mathrm{F}}^2.$$

Each row of $\mathbf{Z}$ selects one prototype: this is a compressed representation.

Issues with K-means: what does distance miss?

- Equal Euclidean distance treats all directions alike.
  Elongated groups may need different spreads and orientations.

Issues with K-means: what does distance miss?

- Equal Euclidean distance treats all directions alike.
  Elongated groups may need different spreads and orientations.
- A point near a boundary still gets a definite assignment.
  Can we express uncertainty about its group?

Issues with K-means: what does distance miss?

- Equal Euclidean distance treats all directions alike.
  Elongated groups may need different spreads and orientations.
- A point near a boundary still gets a definite assignment.
  Can we express uncertainty about its group?
- Feature scales affect distances; standardize when appropriate.

Issues with K-means: what does distance miss?

- Equal Euclidean distance treats all directions alike.
  Elongated groups may need different spreads and orientations.
- A point near a boundary still gets a definite assignment.
  Can we express uncertainty about its group?
- Feature scales affect distances; standardize when appropriate.
- One assignment pass costs $\mathcal{O}(NKD)$; K and initialization matter.

Table of Contents

1 Complete the generative classification story
- GDA: overview and maximum likelihood
- Classifying emails: build a model from the data

2 Unsupervised Learning
- Introduction to Unsupervised Learning
- Clustering and K-means Methods
- Gaussian Mixture Model (GMM)
- Expectation-Maximization Algorithm

Gaussian Mixture Model: a familiar model with hidden labels

In Gaussian discriminant analysis, we knew the class of every training input.

Gaussian Mixture Model: a familiar model with hidden labels

In Gaussian discriminant analysis, we knew the class of every training input.

Now imagine the same Gaussian groups, with the labels erased.

Gaussian Mixture Model: a familiar model with hidden labels

In Gaussian discriminant analysis, we knew the class of every training input.

Now imagine the same Gaussian groups, with the labels erased.

$$z_n \sim \text{Categorical}(\pi_1, \ldots, \pi_K), \qquad \mathbf{x}_n \mid z_n = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Gaussian Mixture Model: a familiar model with hidden labels

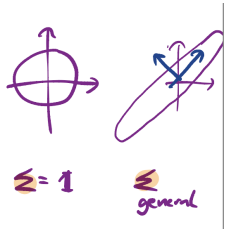
In Gaussian discriminant analysis, we knew the class of every training input.

Now imagine the same Gaussian groups, with the labels erased.

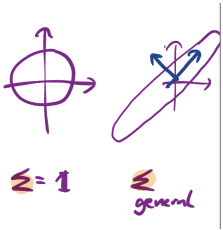
$$z_n \sim \text{Categorical}(\pi_1, \ldots, \pi_K), \qquad \mathbf{x}_n \mid z_n = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Learn centers $\boldsymbol{\mu}_k$, spreads $\boldsymbol{\Sigma}_k$, and proportions π_k from inputs alone.

Two changes from K-means

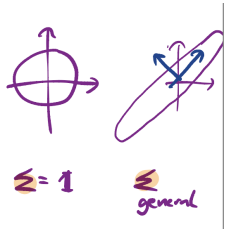


Two changes from K-means



- Replace one common Euclidean geometry by a covariance for each component.

Two changes from K-means



- Replace one common Euclidean geometry by a covariance for each component.
- Replace a definite assignment by a posterior probability for each component.

A GMM models both shape and assignment uncertainty.

Clustering with Gaussians: center, shape, and density

For component k, $\boldsymbol{\mu}_k$ is the center and $\boldsymbol{\Sigma}_k$ sets spread and orientation.

Clustering with Gaussians: center, shape, and density

For component k, $\boldsymbol{\mu}_k$ is the center and $\boldsymbol{\Sigma}_k$ sets spread and orientation.

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k).$$

Clustering with Gaussians: center, shape, and density

For component k, $\boldsymbol{\mu}_k$ is the center and $\boldsymbol{\Sigma}_k$ sets spread and orientation.

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k).$$

- Quadratic term: how far is the input, relative to the component's spread?

Clustering with Gaussians: center, shape, and density

For component k, $\boldsymbol{\mu}_k$ is the center and $\boldsymbol{\Sigma}_k$ sets spread and orientation.

$$\log \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k).$$

- Quadratic term: how far is the input, relative to the component's spread?
- Determinant term: a more spread-out component has a lower peak.

Assume positive definite covariances so the density is well defined.

Soft-clustering: hidden membership, uncertain inference

Each input has one hidden component index $z_n \in \{1, \ldots, K\}$.

$$p(z_n = k) = \pi_k, \qquad \pi_k > 0, \quad \sum_k \pi_k = 1.$$

Soft-clustering: hidden membership, uncertain inference

Each input has one hidden component index $z_n \in \{1, \ldots, K\}$.

$$p(z_n = k) = \pi_k, \qquad \pi_k > 0, \quad \sum_k \pi_k = 1.$$

After observing $\mathbf{x}_n$, ask how plausible each component is:

$$q_{kn} = p(z_n = k \mid \mathbf{x}_n, \boldsymbol{\theta}), \qquad \sum_k q_{kn} = 1.$$

Soft-clustering: hidden membership, uncertain inference

Each input has one hidden component index $z_n \in \{1, \ldots, K\}$.

$$p(z_n = k) = \pi_k, \qquad \pi_k > 0, \quad \sum_k \pi_k = 1.$$

After observing $\mathbf{x}_n$, ask how plausible each component is:

$$q_{kn} = p(z_n = k \mid \mathbf{x}_n, \boldsymbol{\theta}), \qquad \sum_k q_{kn} = 1.$$

For example, $(q_{1n}, q_{2n}) = (0.6, 0.4)$ expresses uncertainty.
It does not mean the point was generated twice.

Gaussian mixture model: joint probability first

For one input and one possible component:

$$p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Gaussian mixture model: joint probability first

For one input and one possible component:

$$p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

The one-hot variable writes the selected term as a product:

$$p(\mathbf{x}_n, \mathbf{z}_n \mid \boldsymbol{\theta}) = \prod_k [\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]^{z_{nk}}.$$

Gaussian mixture model: joint probability first

For one input and one possible component:

$$p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

The one-hot variable writes the selected term as a product:

$$p(\mathbf{x}_n, \mathbf{z}_n \mid \boldsymbol{\theta}) = \prod_k [\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]^{z_{nk}}.$$

For independent inputs, multiply over n; then take logs:

$$\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = \sum_{n,k} z_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

If memberships were observed, we could fit each Gaussian separately.

Marginal likelihood: add the hidden possibilities

We observe $\mathbf{x}_n$, but not which component generated it.

Marginal likelihood: add the hidden possibilities

We observe $\mathbf{x}_n$, but not which component generated it.

$$p(\mathbf{x}_n \mid \boldsymbol{\theta}) = \sum_{k=1}^{K} p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta})$$

Marginal likelihood: add the hidden possibilities

We observe $\mathbf{x}_n$, but not which component generated it.

$$
\begin{aligned}
p(\mathbf{x}_n \mid \boldsymbol{\theta}) &= \sum_{k=1}^{K} p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) \\
&= \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).
\end{aligned}
$$

Marginal likelihood: add the hidden possibilities

We observe $\mathbf{x}_n$, but not which component generated it.

$$
\begin{aligned}
p(\mathbf{x}_n \mid \boldsymbol{\theta}) &= \sum_{k=1}^{K} p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) \\
&= \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).
\end{aligned}
$$

Sum over components for one input; multiply over independent inputs.

$$
p(\mathbf{X} \mid \boldsymbol{\theta}) = \prod_{n=1}^{N} p(\mathbf{x}_n \mid \boldsymbol{\theta}).
$$

Maximum likelihood: where fitting becomes coupled

$$\max_{\boldsymbol{\theta}} \ \ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

Maximum likelihood: where fitting becomes coupled

$$\max_{\boldsymbol{\theta}} \ \ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

The log is outside the sum: we cannot fit each Gaussian independently.

Maximum likelihood: where fitting becomes coupled

$$\max_{\boldsymbol{\theta}} \ \ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

The log is outside the sum: we cannot fit each Gaussian independently.

Known memberships $\Rightarrow$ separate Gaussian fits.
Unknown memberships $\Rightarrow$ fitting and assignment depend on each other.

Maximum likelihood: where fitting becomes coupled

$$\max_{\boldsymbol{\theta}} \ \ell(\boldsymbol{\theta}) = \sum_{n=1}^{N} \log \left[\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

The log is outside the sum: we cannot fit each Gaussian independently.

Known memberships $\Rightarrow$ separate Gaussian fits.
Unknown memberships $\Rightarrow$ fitting and assignment depend on each other.

EM alternates between inferring memberships and fitting weighted Gaussians.

Bridge: what if the missing labels were supplied?

Let $N_k = \sum_n z_{nk}$ be the number of inputs in group k.

Bridge: what if the missing labels were supplied?

Let $N_k = \sum_n z_{nk}$ be the number of inputs in group k.

$$\widehat{\pi}_k = \frac{N_k}{N}, \qquad \widehat{\boldsymbol{\mu}}_k = \frac{\sum_n z_{nk} \mathbf{x}_n}{N_k}.$$

Bridge: what if the missing labels were supplied?

Let $N_k = \sum_n z_{nk}$ be the number of inputs in group k.

$$\widehat{\pi}_k = \frac{N_k}{N}, \qquad \widehat{\boldsymbol{\mu}}_k = \frac{\sum_n z_{nk} \mathbf{x}_n}{N_k}.$$

$$\widehat{\boldsymbol{\Sigma}}_k = \frac{\sum_n z_{nk} (\mathbf{x}_n - \widehat{\boldsymbol{\mu}}_k)(\mathbf{x}_n - \widehat{\boldsymbol{\mu}}_k)^\top}{N_k}.$$

Bridge: what if the missing labels were supplied?

Let $N_k = \sum_n z_{nk}$ be the number of inputs in group k.

$$\widehat{\pi}_k = \frac{N_k}{N}, \qquad \widehat{\boldsymbol{\mu}}_k = \frac{\sum_n z_{nk} \mathbf{x}_n}{N_k}.$$

$$\widehat{\boldsymbol{\Sigma}}_k = \frac{\sum_n z_{nk} (\mathbf{x}_n - \widehat{\boldsymbol{\mu}}_k)(\mathbf{x}_n - \widehat{\boldsymbol{\mu}}_k)^\top}{N_k}.$$

These are the count, center, and spread estimates from our opening GDA example.
EM will replace the unknown $0/1$ indicators by their posterior averages.

Table of Contents

1 Complete the generative classification story
- GDA: overview and maximum likelihood
- Classifying emails: build a model from the data

2 Unsupervised Learning
- Introduction to Unsupervised Learning
- Clustering and K-means Methods
- Gaussian Mixture Model (GMM)
- Expectation-Maximization Algorithm

Motivation of EM: the labels are the missing piece

For a GMM, $p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

$$\ell(\boldsymbol{\theta}) = \sum_n \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Motivation of EM: the labels are the missing piece

For a GMM, $p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

$$\ell(\boldsymbol{\theta}) = \sum_n \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

If we knew the component labels, we could count and fit each group.

Motivation of EM: the labels are the missing piece

For a GMM, $p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

$$\ell(\boldsymbol{\theta}) = \sum_n \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

If we knew the component labels, we could count and fit each group.

But membership depends on the parameters we are trying to learn.

Motivation of EM: the labels are the missing piece

For a GMM, $p(\mathbf{x}_n, z_n = k \mid \boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

$$\ell(\boldsymbol{\theta}) = \sum_n \log \sum_k \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

If we knew the component labels, we could count and fit each group.

But membership depends on the parameters we are trying to learn.

EM: infer probabilities with the current model, then refit using those probabilities.

Notation: an index is different from an indicator

For $K = 3$, suppose input n came from component 2.

$$z_n = 2 \qquad \Longleftrightarrow \qquad \mathbf{z}_n = (0, 1, 0)^\top.$$

Notation: an index is different from an indicator

For $K = 3$, suppose input n came from component 2.

$$z_n = 2 \qquad \Longleftrightarrow \qquad \mathbf{z}_n = (0, 1, 0)^\top.$$

Now suppose we are uncertain about its origin:

$$(q_{1n}, q_{2n}, q_{3n}) = (0.2, 0.7, 0.1).$$

Notation: an index is different from an indicator

For $K = 3$, suppose input n came from component 2.

$$z_n = 2 \qquad \Longleftrightarrow \qquad \mathbf{z}_n = (0, 1, 0)^\top.$$

Now suppose we are uncertain about its origin:

$$(q_{1n}, q_{2n}, q_{3n}) = (0.2, 0.7, 0.1).$$

$q_{kn} = p(z_n = k \mid \mathbf{x}_n, \boldsymbol{\theta})$ is a probability, not a component index.

$$\mathbb{E}[z_{nk} \mid \mathbf{x}_n, \boldsymbol{\theta}] = q_{kn}.$$

An Overview of EM: freeze, infer, refit

1. Start with parameters $\boldsymbol{\theta}^{(t)}$.

An Overview of EM: freeze, infer, refit

1. Start with parameters $\boldsymbol{\theta}^{(t)}$.
2. **E-step: freeze the parameters.**
   Find responsibilities $q_{kn}^{(t)}$ for every input and component.

An Overview of EM: freeze, infer, refit

1. Start with parameters $\boldsymbol{\theta}^{(t)}$.
2. **E-step: freeze the parameters.**
   Find responsibilities $q_{kn}^{(t)}$ for every input and component.
3. **M-step: freeze the responsibilities.**
   Update counts, centers, and spreads to obtain $\boldsymbol{\theta}^{(t+1)}$.

An Overview of EM: freeze, infer, refit

1. Start with parameters $\boldsymbol{\theta}^{(t)}$.
2. **E-step: freeze the parameters.**
   Find responsibilities $q_{kn}^{(t)}$ for every input and component.
3. **M-step: freeze the responsibilities.**
   Update counts, centers, and spreads to obtain $\boldsymbol{\theta}^{(t+1)}$.

Then recompute responsibilities. Old probabilities need not fit the new model.

The E-step: Bayes rule, one input at a time

Keep $\boldsymbol{\theta}^{(t)}$ fixed and observe one input $\mathbf{x}_n$.

The E-step: Bayes rule, one input at a time

Keep $\boldsymbol{\theta}^{(t)}$ fixed and observe one input $\mathbf{x}_n$.

$$a_{kn}^{(t)} = \underbrace{\pi_k^{(t)}}_{\text{prior weight}} \underbrace{\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}_{\text{component density}}.$$

The E-step: Bayes rule, one input at a time

Keep $\boldsymbol{\theta}^{(t)}$ fixed and observe one input $\mathbf{x}_n$.

$$a_{kn}^{(t)} = \underbrace{\pi_k^{(t)}}_{\text{prior weight}} \underbrace{\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}_{\text{component density}}.$$

Normalize across the possible components for this input:

$$q_{kn}^{(t)} = \frac{a_{kn}^{(t)}}{\sum_{j=1}^{K} a_{jn}^{(t)}} = p(z_n = k \mid \mathbf{x}_n, \boldsymbol{\theta}^{(t)}).$$

The E-step: Bayes rule, one input at a time

Keep $\boldsymbol{\theta}^{(t)}$ fixed and observe one input $\mathbf{x}_n$.

$$a_{kn}^{(t)} = \underbrace{\pi_k^{(t)}}_{\text{prior weight}} \underbrace{\mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}_{\text{component density}}.$$

Normalize across the possible components for this input:

$$q_{kn}^{(t)} = \frac{a_{kn}^{(t)}}{\sum_{j=1}^{K} a_{jn}^{(t)}} = p(z_n = k \mid \mathbf{x}_n, \boldsymbol{\theta}^{(t)}).$$

Check: $q_{kn}^{(t)} \geq 0$ and $\sum_k q_{kn}^{(t)} = 1$ for every n.

Bridge: E-step on the same three inputs

Use $x = (0, 2, 4)$, $\mu_1 = 0$, $\mu_2 = 4$, $\sigma_1^2 = \sigma_2^2 = 4$, and $\pi_1 = \pi_2 = 1/2$.

x_n	0	2	4

Bridge: E-step on the same three inputs

Use $x = (0, 2, 4)$, $\mu_1 = 0$, $\mu_2 = 4$, $\sigma_1^2 = \sigma_2^2 = 4$, and $\pi_1 = \pi_2 = 1/2$.

$$\frac{q_{1n}}{q_{2n}} = \exp\left(\frac{(x_n - 4)^2 - x_n^2}{8}\right) = e^{2 - x_n}.$$

x_n	0	2	4

Bridge: E-step on the same three inputs

Use $x = (0, 2, 4)$, $\mu_1 = 0$, $\mu_2 = 4$, $\sigma_1^2 = \sigma_2^2 = 4$, and $\pi_1 = \pi_2 = 1/2$.

$$\frac{q_{1n}}{q_{2n}} = \exp\left(\frac{(x_n - 4)^2 - x_n^2}{8}\right) = e^{2 - x_n}.$$

x_n	0	2	4
q_{1n}	0.881	0.500	0.119
q_{2n}	0.119	0.500	0.881

Bridge: E-step on the same three inputs

Use $x = (0, 2, 4)$, $\mu_1 = 0$, $\mu_2 = 4$, $\sigma_1^2 = \sigma_2^2 = 4$, and $\pi_1 = \pi_2 = 1/2$.

$$\frac{q_{1n}}{q_{2n}} = \exp\left(\frac{(x_n - 4)^2 - x_n^2}{8}\right) = e^{2 - x_n}.$$

x_n	0	2	4
q_{1n}	0.881	0.500	0.119
q_{2n}	0.119	0.500	0.881

The middle input shares its weight equally. The endpoints remain uncertain.

The M-step: count fractions of inputs

Freeze $q_{kn}^{(t)}$. Define the effective count for component k:

The M-step: count fractions of inputs

Freeze $q_{kn}^{(t)}$. Define the effective count for component k:

$$N_k^{(t)} = \sum_{n=1}^{N} q_{kn}^{(t)}.$$

The M-step: count fractions of inputs

Freeze $q_{kn}^{(t)}$. Define the effective count for component k:

$$N_k^{(t)} = \sum_{n=1}^{N} q_{kn}^{(t)}.$$

In our example: $N_1 = 0.881 + 0.500 + 0.119 = 1.500$ and $N_2 = 1.500$.

The M-step: count fractions of inputs

Freeze $q_{kn}^{(t)}$. Define the effective count for component k:

$$N_k^{(t)} = \sum_{n=1}^{N} q_{kn}^{(t)}.$$

In our example: $N_1 = 0.881 + 0.500 + 0.119 = 1.500$ and $N_2 = 1.500$.

$$\sum_k N_k^{(t)} = \sum_n \underbrace{\sum_k q_{kn}^{(t)}}_{1} = N.$$

Each input contributes a total weight of one, distributed across components.

The M-step: weighted center and mixture proportion

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} \mathbf{x}_n}{N_k^{(t)}}, \qquad \pi_k^{(t+1)} = \frac{N_k^{(t)}}{N}.$$

The M-step: weighted center and mixture proportion

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} \mathbf{x}_n}{N_k^{(t)}}, \qquad \pi_k^{(t+1)} = \frac{N_k^{(t)}}{N}.$$

For component 1 in the same example:

$$\mu_1^{\text{new}} \approx \frac{0.8808 \cdot 0 + 0.5000 \cdot 2 + 0.1192 \cdot 4}{1.500} \approx 0.985.$$

The M-step: weighted center and mixture proportion

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} \mathbf{x}_n}{N_k^{(t)}}, \qquad \pi_k^{(t+1)} = \frac{N_k^{(t)}}{N}.$$

For component 1 in the same example:

$$\mu_1^{\text{new}} \approx \frac{0.8808 \cdot 0 + 0.5000 \cdot 2 + 0.1192 \cdot 4}{1.500} \approx 0.985.$$

$$\mu_2^{\text{new}} \approx 3.015, \qquad \pi_1^{\text{new}} = \pi_2^{\text{new}} = 0.5.$$

The M-step: weighted center and mixture proportion

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} \mathbf{x}_n}{N_k^{(t)}}, \qquad \pi_k^{(t+1)} = \frac{N_k^{(t)}}{N}.$$

For component 1 in the same example:

$$\mu_1^{\text{new}} \approx \frac{0.8808 \cdot 0 + 0.5000 \cdot 2 + 0.1192 \cdot 4}{1.500} \approx 0.985.$$

$$\mu_2^{\text{new}} \approx 3.015, \qquad \pi_1^{\text{new}} = \pi_2^{\text{new}} = 0.5.$$

K-means uses $0/1$ counts; EM uses posterior weights in the same averaging operation.

The M-step: spread around the new center

Freeze the same $q_{kn}^{(t)}$, and use the **new** mean:

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top}{N_k^{(t)}}.$$

The M-step: spread around the new center

Freeze the same $q_{kn}^{(t)}$, and use the **new** mean:

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top}{N_k^{(t)}}.$$

In one dimension, this is just a weighted average of squared residuals:

$$(\sigma_1^2)^{\text{new}} \approx \frac{0.8808(0 - 0.985)^2 + 0.5(2 - 0.985)^2 + 0.1192(4 - 0.985)^2}{1.5} \approx 1.636.$$

The M-step: spread around the new center

Freeze the same $q_{kn}^{(t)}$, and use the **new** mean:

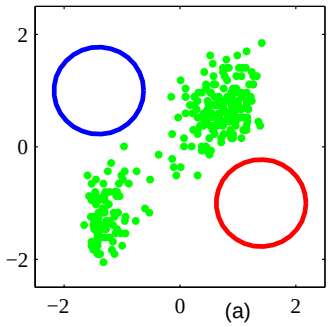
$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top}{N_k^{(t)}}.$$

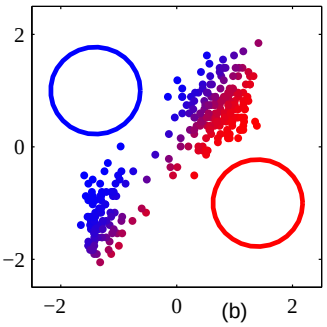
In one dimension, this is just a weighted average of squared residuals:

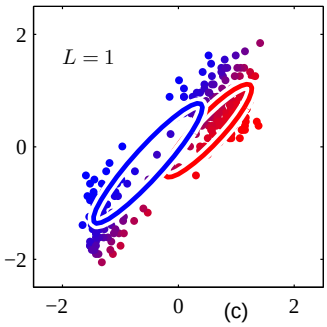
$$(\sigma_1^2)^{\text{new}} \approx \frac{0.8808(0 - 0.985)^2 + 0.5(2 - 0.985)^2 + 0.1192(4 - 0.985)^2}{1.5} \approx 1.636.$$

Symmetry gives $(\sigma_2^2)^{\text{new}} \approx 1.636$.
Now repeat the E-step with the new centers, variances, and proportions.

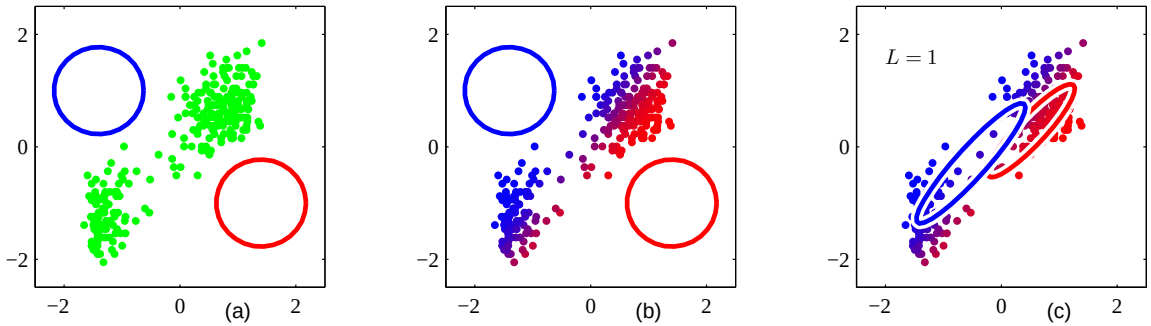
EM for GMM: watch assignment and shape change





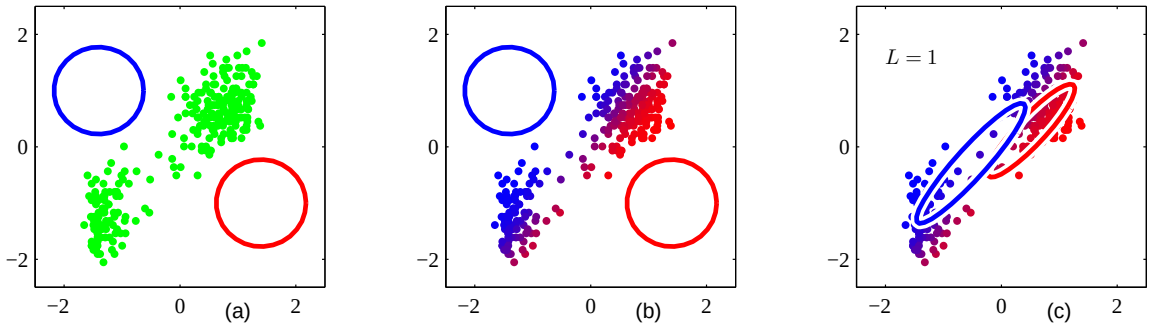


EM for GMM: watch assignment and shape change



Left: initial Gaussians. Middle: infer memberships. Right: refit the Gaussians.

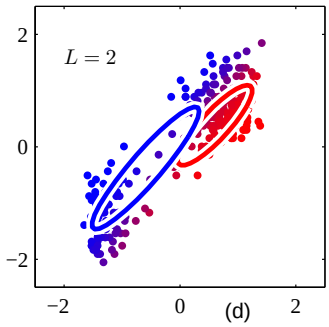
EM for GMM: watch assignment and shape change

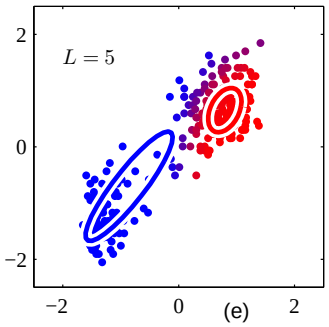


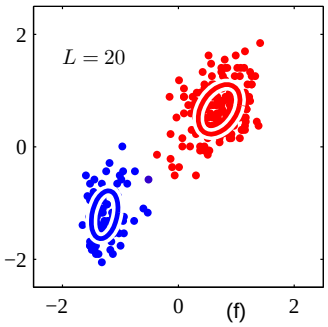
Left: initial Gaussians. Middle: infer memberships. Right: refit the Gaussians.

The parameters move because each input contributes in proportion to its responsibility.

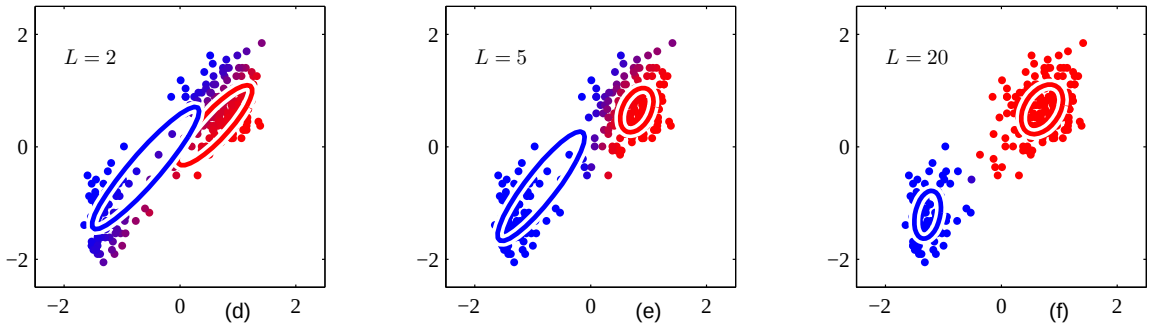
EM for GMM: repeat with the updated model





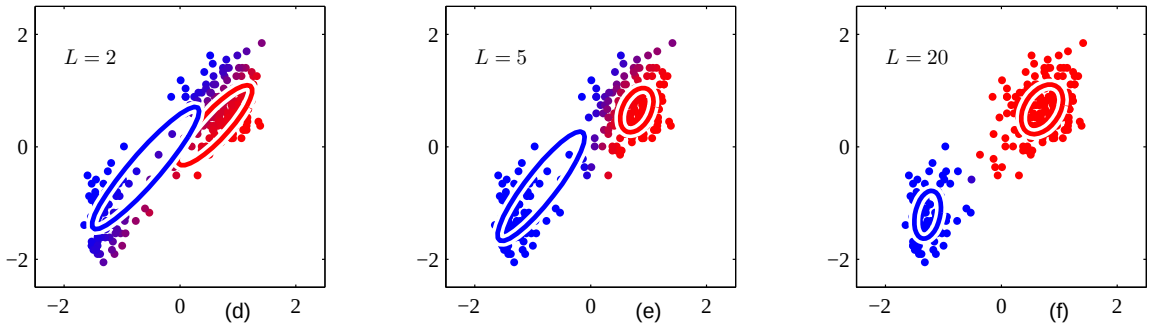


EM for GMM: repeat with the updated model



New parameters change the responsibilities; new responsibilities change the fit.

EM for GMM: repeat with the updated model



New parameters change the responsibilities; new responsibilities change the fit.

We can now run EM. Next: what objective makes these updates sensible?

M-step objective: average over the missing labels

With known labels, the complete-data log-likelihood is

$$\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = \sum_{n,k} z_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

M-step objective: average over the missing labels

With known labels, the complete-data log-likelihood is

$$\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = \sum_{n,k} z_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

Average this expression using the **old model's** posterior:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \sum_{n,k} q_{kn}^{(t)} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

M-step objective: average over the missing labels

With known labels, the complete-data log-likelihood is

$$\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = \sum_{n,k} z_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

Average this expression using the **old model's** posterior:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \sum_{n,k} q_{kn}^{(t)} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

Because $\mathbb{E}[z_{nk} \mid \mathbf{X}, \boldsymbol{\theta}^{(t)}] = q_{kn}^{(t)}$, indicators become weights.

M-step objective: average over the missing labels

With known labels, the complete-data log-likelihood is

$$\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = \sum_{n,k} z_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

Average this expression using the **old model's** posterior:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \sum_{n,k} q_{kn}^{(t)} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)].$$

Because $\mathbb{E}[z_{nk} \mid \mathbf{X}, \boldsymbol{\theta}^{(t)}] = q_{kn}^{(t)}$, indicators become weights.

The M-step maximizes Q over $\boldsymbol{\theta}$ while keeping those weights fixed.

M-step Derivation: the weighted mean

Retain only the mean-dependent terms of Q:

$$Q_k(\boldsymbol{\mu}_k) = -\frac{1}{2}\sum_n q_{kn}^{(t)}(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) + \text{const.}$$

M-step Derivation: the weighted mean

Retain only the mean-dependent terms of Q:

$$Q_k(\boldsymbol{\mu}_k) = -\frac{1}{2}\sum_n q_{kn}^{(t)}(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) + \text{const.}$$

Differentiate, set to zero, and multiply by $\boldsymbol{\Sigma}_k$:

$$\sum_n q_{kn}^{(t)} \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) = 0 \implies \sum_n q_{kn}^{(t)} \mathbf{x}_n = N_k^{(t)} \boldsymbol{\mu}_k.$$

M-step Derivation: the weighted mean

Retain only the mean-dependent terms of Q:

$$Q_k(\boldsymbol{\mu}_k) = -\frac{1}{2}\sum_n q_{kn}^{(t)}(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) + \text{const.}$$

Differentiate, set to zero, and multiply by $\boldsymbol{\Sigma}_k$:

$$\sum_n q_{kn}^{(t)} \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) = 0 \implies \sum_n q_{kn}^{(t)} \mathbf{x}_n = N_k^{(t)} \boldsymbol{\mu}_k.$$

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_n q_{kn}^{(t)} \mathbf{x}_n}{N_k^{(t)}}.$$

Same stationary equation as K-means, with fractional weights.

M-step Derivation: the weighted covariance

Use the new mean. Write $S_k = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top$.

M-step Derivation: the weighted covariance

Use the new mean. Write $S_k = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top$.

$$\boldsymbol{\Lambda}_k = \boldsymbol{\Sigma}_k^{-1}, \qquad Q_k = \frac{N_k^{(t)}}{2} \log |\boldsymbol{\Lambda}_k| - \frac{1}{2} \operatorname{tr}(\boldsymbol{\Lambda}_k S_k) + \text{const.}$$

M-step Derivation: the weighted covariance

Use the new mean. Write $S_k = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top$.

$$\boldsymbol{\Lambda}_k = \boldsymbol{\Sigma}_k^{-1}, \qquad Q_k = \frac{N_k^{(t)}}{2} \log |\boldsymbol{\Lambda}_k| - \frac{1}{2} \operatorname{tr}(\boldsymbol{\Lambda}_k S_k) + \text{const.}$$

Use $\nabla_{\boldsymbol{\Lambda}} \log |\boldsymbol{\Lambda}| = \boldsymbol{\Lambda}^{-1}$ for symmetric positive definite $\boldsymbol{\Lambda}$:

$$\frac{N_k^{(t)}}{2} \boldsymbol{\Lambda}_k^{-1} - \frac{1}{2} S_k = 0 \quad \Longrightarrow \quad \boldsymbol{\Sigma}_k^{(t+1)} = \frac{S_k}{N_k^{(t)}}.$$

M-step Derivation: the weighted covariance

Use the new mean. Write $S_k = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top$.

$$\boldsymbol{\Lambda}_k = \boldsymbol{\Sigma}_k^{-1}, \qquad Q_k = \frac{N_k^{(t)}}{2} \log |\boldsymbol{\Lambda}_k| - \frac{1}{2} \operatorname{tr}(\boldsymbol{\Lambda}_k S_k) + \text{const.}$$

Use $\nabla_{\boldsymbol{\Lambda}} \log |\boldsymbol{\Lambda}| = \boldsymbol{\Lambda}^{-1}$ for symmetric positive definite $\boldsymbol{\Lambda}$:

$$\frac{N_k^{(t)}}{2} \boldsymbol{\Lambda}_k^{-1} - \frac{1}{2} S_k = 0 \quad \Longrightarrow \quad \boldsymbol{\Sigma}_k^{(t+1)} = \frac{S_k}{N_k^{(t)}}.$$

If S_k is singular, there is no positive definite interior solution.

M-step Derivation: the mixture proportions

Maximize $\sum_k N_k^{(t)} \log \pi_k$ subject to $\sum_k \pi_k = 1$.

M-step Derivation: the mixture proportions

Maximize $\sum_k N_k^{(t)} \log \pi_k$ subject to $\sum_k \pi_k = 1$.

$$\mathcal{J} = \sum_k N_k^{(t)} \log \pi_k - \lambda(\sum_k \pi_k - 1).$$

M-step Derivation: the mixture proportions

Maximize $\sum_k N_k^{(t)} \log \pi_k$ subject to $\sum_k \pi_k = 1$.

$$\mathcal{J} = \sum_k N_k^{(t)} \log \pi_k - \lambda(\sum_k \pi_k - 1).$$

$$\frac{N_k^{(t)}}{\pi_k} - \lambda = 0 \quad \Longrightarrow \quad N_k^{(t)} = \lambda \pi_k.$$

M-step Derivation: the mixture proportions

Maximize $\sum_k N_k^{(t)} \log \pi_k$ subject to $\sum_k \pi_k = 1$.

$$\mathcal{J} = \sum_k N_k^{(t)} \log \pi_k - \lambda(\sum_k \pi_k - 1).$$

$$\frac{N_k^{(t)}}{\pi_k} - \lambda = 0 \quad \Longrightarrow \quad N_k^{(t)} = \lambda \pi_k.$$

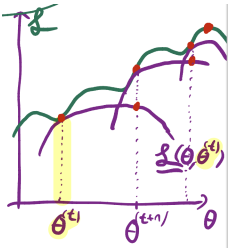
Sum over k: $N = \lambda$, because $\sum_k N_k^{(t)} = N$.

$$\pi_k^{(t+1)} = \frac{N_k^{(t)}}{N}.$$

A component's mixture proportion is its share of the total effective count.

Why can maximizing Q help the observed likelihood?

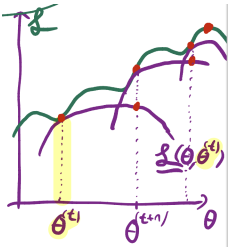
We want to improve $\ell(\boldsymbol{\theta})$, but we just optimized $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$.



Why can maximizing Q help the observed likelihood?

We want to improve $\ell(\boldsymbol{\theta})$, but we just optimized $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$.

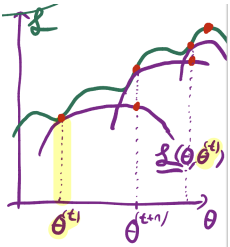
The connection is a lower bound that touches ℓ at the current parameters.



Why can maximizing Q help the observed likelihood?

We want to improve $\ell(\boldsymbol{\theta})$, but we just optimized $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$.

The connection is a lower bound that touches ℓ at the current parameters.



Raise this touching bound $\Rightarrow$ the true likelihood cannot decrease.
We will build the bound one input at a time.

Build the lower bound: insert weights, then use log concavity

For one input, let $a_k(\boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Choose $q_k > 0$ with $\sum_k q_k = 1$.

Build the lower bound: insert weights, then use log concavity

For one input, let $a_k(\boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Choose $q_k > 0$ with $\sum_k q_k = 1$.

$$\log \sum_k a_k(\boldsymbol{\theta}) = \log \sum_k q_k \frac{a_k(\boldsymbol{\theta})}{q_k}$$

Build the lower bound: insert weights, then use log concavity

For one input, let $a_k(\boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Choose $q_k > 0$ with $\sum_k q_k = 1$.

$$\log \sum_k a_k(\boldsymbol{\theta}) = \log \sum_k q_k \frac{a_k(\boldsymbol{\theta})}{q_k}$$

$$\geq \sum_k q_k \log \frac{a_k(\boldsymbol{\theta})}{q_k}.$$

This is Jensen's inequality: $\log(\text{weighted average}) \geq$ weighted average of logs.

Build the lower bound: insert weights, then use log concavity

For one input, let $a_k(\boldsymbol{\theta}) = \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Choose $q_k > 0$ with $\sum_k q_k = 1$.

$$\log \sum_k a_k(\boldsymbol{\theta}) = \log \sum_k q_k \frac{a_k(\boldsymbol{\theta})}{q_k}$$

$$\geq \sum_k q_k \log \frac{a_k(\boldsymbol{\theta})}{q_k}.$$

This is Jensen's inequality: $\log(\text{weighted average}) \geq$ weighted average of logs.

Equality holds when all ratios $a_k(\boldsymbol{\theta})/q_k$ are equal.

Make the bound tight: recover the E-step

At $\boldsymbol{\theta}^{(t)}$, equality requires $a_k(\boldsymbol{\theta}^{(t)})/q_k = c$ for every k.

Make the bound tight: recover the E-step

At $\boldsymbol{\theta}^{(t)}$, equality requires $a_k(\boldsymbol{\theta}^{(t)})/q_k = c$ for every k.

$$q_k = \frac{a_k(\boldsymbol{\theta}^{(t)})}{c}, \qquad 1 = \sum_k q_k = \frac{\sum_k a_k(\boldsymbol{\theta}^{(t)})}{c}.$$

Make the bound tight: recover the E-step

At $\boldsymbol{\theta}^{(t)}$, equality requires $a_k(\boldsymbol{\theta}^{(t)})/q_k = c$ for every k.

$$q_k = \frac{a_k(\boldsymbol{\theta}^{(t)})}{c}, \qquad 1 = \sum_k q_k = \frac{\sum_k a_k(\boldsymbol{\theta}^{(t)})}{c}.$$

Thus $c = \sum_j a_j(\boldsymbol{\theta}^{(t)})$, giving

$$q_{kn}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_j \pi_j^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}.$$

Make the bound tight: recover the E-step

At $\boldsymbol{\theta}^{(t)}$, equality requires $a_k(\boldsymbol{\theta}^{(t)})/q_k = c$ for every k.

$$q_k = \frac{a_k(\boldsymbol{\theta}^{(t)})}{c}, \qquad 1 = \sum_k q_k = \frac{\sum_k a_k(\boldsymbol{\theta}^{(t)})}{c}.$$

Thus $c = \sum_j a_j(\boldsymbol{\theta}^{(t)})$, giving

$$q_{kn}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_j \pi_j^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}.$$

The posterior probabilities we already computed make the bound touch the likelihood.

The Evidence Lower Bound: expected fit plus entropy

Sum the per-input bounds. Write $q_{kn} = q_n(z_n = k)$:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \sum_{n,k} q_{kn} \log \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{q_{kn}} \leq \ell(\boldsymbol{\theta}).$$

The Evidence Lower Bound: expected fit plus entropy

Sum the per-input bounds. Write $q_{kn} = q_n(z_n = k)$:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \sum_{n,k} q_{kn} \log \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{q_{kn}} \leq \ell(\boldsymbol{\theta}).$$

Split the log ratio:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \underbrace{\sum_{n,k} q_{kn} \log[\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]}_{\text{expected complete-data log-likelihood}} + \underbrace{\left(- \sum_{n,k} q_{kn} \log q_{kn} \right)}_{H(q)\text{: entropy}}.$$

The Evidence Lower Bound: expected fit plus entropy

Sum the per-input bounds. Write $q_{kn} = q_n(z_n = k)$:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \sum_{n,k} q_{kn} \log \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{q_{kn}} \leq \ell(\boldsymbol{\theta}).$$

Split the log ratio:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \underbrace{\sum_{n,k} q_{kn} \log[\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]}_{\text{expected complete-data log-likelihood}} + \underbrace{\left(- \sum_{n,k} q_{kn} \log q_{kn} \right)}_{H(q)\text{: entropy}}.$$

In the M-step, $q = q^{(t)}$ is fixed, so $H(q^{(t)})$ is constant.
Maximizing $\mathcal{F}$ over $\boldsymbol{\theta}$ is exactly maximizing Q.

EM algorithm: the three-line improvement argument

E-step: $q^{(t)}$ is the posterior under $\boldsymbol{\theta}^{(t)}$.

$$\ell(\boldsymbol{\theta}^{(t)}) = \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

EM algorithm: the three-line improvement argument

E-step: $q^{(t)}$ is the posterior under $\boldsymbol{\theta}^{(t)}$.

$$\ell(\boldsymbol{\theta}^{(t)}) = \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

M-step: raise the bound while keeping $q^{(t)}$ fixed.

$$\mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

EM algorithm: the three-line improvement argument

E-step: $q^{(t)}$ is the posterior under $\boldsymbol{\theta}^{(t)}$.

$$\ell(\boldsymbol{\theta}^{(t)}) = \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

M-step: raise the bound while keeping $q^{(t)}$ fixed.

$$\mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

The new likelihood is at least its lower bound.

$$\ell(\boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}) = \ell(\boldsymbol{\theta}^{(t)}).$$

EM algorithm: the three-line improvement argument

E-step: $q^{(t)}$ is the posterior under $\boldsymbol{\theta}^{(t)}$.

$$\ell(\boldsymbol{\theta}^{(t)}) = \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

M-step: raise the bound while keeping $q^{(t)}$ fixed.

$$\mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}).$$

The new likelihood is at least its lower bound.

$$\ell(\boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t+1)}) \geq \mathcal{F}(q^{(t)}, \boldsymbol{\theta}^{(t)}) = \ell(\boldsymbol{\theta}^{(t)}).$$

Touch the old value $\rightarrow$ raise the bound $\rightarrow$ reach a likelihood no smaller.

Background: KL measures the gap between distributions

For discrete probabilities on a common support:

$$D_{\mathrm{KL}}(q\|p) = \sum_{z} q(z) \log \frac{q(z)}{p(z)}.$$

Background: KL measures the gap between distributions

For discrete probabilities on a common support:

$$D_{\mathrm{KL}}(q\|p) = \sum_z q(z) \log \frac{q(z)}{p(z)}.$$

- $D_{\mathrm{KL}}(q\|p) \geq 0$; equality holds exactly when $q = p$.

Background: KL measures the gap between distributions

For discrete probabilities on a common support:

$$D_{\mathrm{KL}}(q\|p) = \sum_{z} q(z) \log \frac{q(z)}{p(z)}.$$

- $D_{\mathrm{KL}}(q\|p) \geq 0$; equality holds exactly when $q = p$.
- It is generally asymmetric: exchanging q and p changes the value.

Background: KL measures the gap between distributions

For discrete probabilities on a common support:

$$D_{\mathrm{KL}}(q\|p) = \sum_z q(z) \log \frac{q(z)}{p(z)}.$$

- $D_{\mathrm{KL}}(q\|p) \geq 0$; equality holds exactly when $q = p$.
- It is generally asymmetric: exchanging q and p changes the value.

In EM, compare our distribution over hidden labels with the model's posterior.

ELBO: the exact gap is a posterior KL

For $q(\mathbf{Z}) = \prod_n q_n(z_n)$, use $p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}) p(\mathbf{X} \mid \boldsymbol{\theta})$:

ELBO: the exact gap is a posterior KL

For $q(\mathbf{Z}) = \prod_n q_n(z_n)$, use $p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}) p(\mathbf{X} \mid \boldsymbol{\theta})$:

$$\mathcal{F}(q, \boldsymbol{\theta}) = \mathbb{E}_q \left[\log \frac{p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})}{q(\mathbf{Z})} \right]$$

ELBO: the exact gap is a posterior KL

For $q(\mathbf{Z}) = \prod_n q_n(z_n)$, use $p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}) p(\mathbf{X} \mid \boldsymbol{\theta})$:

$$\begin{aligned} \mathcal{F}(q, \boldsymbol{\theta}) &= \mathbb{E}_q \left[\log \frac{p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})}{q(\mathbf{Z})} \right] \\ &= \ell(\boldsymbol{\theta}) + \mathbb{E}_q \left[\log \frac{p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})}{q(\mathbf{Z})} \right] \end{aligned}$$

ELBO: the exact gap is a posterior KL

For $q(\mathbf{Z}) = \prod_n q_n(z_n)$, use $p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) = p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}) p(\mathbf{X} \mid \boldsymbol{\theta})$:

$$\begin{aligned}
\mathcal{F}(q, \boldsymbol{\theta}) &= \mathbb{E}_q \left[\log \frac{p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})}{q(\mathbf{Z})} \right] \\
&= \ell(\boldsymbol{\theta}) + \mathbb{E}_q \left[\log \frac{p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})}{q(\mathbf{Z})} \right] \\
&= \ell(\boldsymbol{\theta}) - D_{\mathrm{KL}} \left(q(\mathbf{Z}) \| p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}) \right).
\end{aligned}$$

E-step: zero gap at the old model. M-step: raise $\mathcal{F}$ with q fixed.

The relationship to K-means: hard and soft assignments

	K-means	GMM with EM
Assignment	$z_{nk} \in \{0, 1\}$	$q_{kn} \in [0, 1]$

The relationship to K-means: hard and soft assignments

	K-means	GMM with EM
Assignment	$z_{nk} \in \{0, 1\}$	$q_{kn} \in [0, 1]$
Count	$\sum_n z_{nk}$	$\sum_n q_{kn}$

The relationship to K-means: hard and soft assignments

	K-means	GMM with EM
Assignment	$z_{nk} \in \{0, 1\}$	$q_{kn} \in [0, 1]$
Count	$\sum_n z_{nk}$	$\sum_n q_{kn}$
Center	mean of assigned points	weighted mean

The relationship to K-means: hard and soft assignments

	K-means	GMM with EM
Assignment	$z_{nk} \in \{0, 1\}$	$q_{kn} \in [0, 1]$
Count	$\sum_n z_{nk}$	$\sum_n q_{kn}$
Center	mean of assigned points	weighted mean
Geometry	Euclidean distance	learned covariance

Both alternate assignment and fitting; their objectives are different.

When does a soft assignment become a nearest-center choice?

Constrain all components to a common fixed $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$.

$$q_{kn} = \frac{\pi_k \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/(2\sigma^2)]}{\sum_j \pi_j \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/(2\sigma^2)]}.$$

When does a soft assignment become a nearest-center choice?

Constrain all components to a common fixed $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$.

$$q_{kn} = \frac{\pi_k \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/(2\sigma^2)]}{\sum_j \pi_j \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/(2\sigma^2)]}.$$

For the unique nearest center k_*, and any other j:

$$\frac{q_{jn}}{q_{k_* n}} = \frac{\pi_j}{\pi_{k_*}} \exp\left[-\frac{\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 - \|\mathbf{x}_n - \boldsymbol{\mu}_{k_*}\|^2}{2\sigma^2}\right] \to 0.$$

When does a soft assignment become a nearest-center choice?

Constrain all components to a common fixed $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$.

$$q_{kn} = \frac{\pi_k \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/(2\sigma^2)]}{\sum_j \pi_j \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/(2\sigma^2)]}.$$

For the unique nearest center k_*, and any other j:

$$\frac{q_{jn}}{q_{k_* n}} = \frac{\pi_j}{\pi_{k_*}} \exp\left[-\frac{\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 - \|\mathbf{x}_n - \boldsymbol{\mu}_{k_*}\|^2}{2\sigma^2}\right] \to 0.$$

As $\sigma^2 \to 0$, assignments become one-hot; mean updates become K-means updates.

When does a soft assignment become a nearest-center choice?

Constrain all components to a common fixed $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}$.

$$q_{kn} = \frac{\pi_k \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/(2\sigma^2)]}{\sum_j \pi_j \exp[-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/(2\sigma^2)]}.$$

For the unique nearest center k_*, and any other j:

$$\frac{q_{jn}}{q_{k_* n}} = \frac{\pi_j}{\pi_{k_*}} \exp\left[-\frac{\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 - \|\mathbf{x}_n - \boldsymbol{\mu}_{k_*}\|^2}{2\sigma^2}\right] \to 0.$$

As $\sigma^2 \to 0$, assignments become one-hot; mean updates become K-means updates.

This uses common *isotropic* covariance and positive fixed proportions.
A diagonal covariance alone is not sufficient.

EM Convergence Properties: what the guarantee says

Exact E-steps and M-steps give $\ell(\boldsymbol{\theta}^{(t+1)}) \geq \ell(\boldsymbol{\theta}^{(t)})$.

EM Convergence Properties: what the guarantee says

Exact E-steps and M-steps give $\ell(\boldsymbol{\theta}^{(t+1)}) \geq \ell(\boldsymbol{\theta}^{(t)})$.

- If the likelihood values are bounded above, these values converge.

EM Convergence Properties: what the guarantee says

Exact E-steps and M-steps give $\ell(\boldsymbol{\theta}^{(t+1)}) \geq \ell(\boldsymbol{\theta}^{(t)})$.

- If the likelihood values are bounded above, these values converge.
- This alone does not guarantee a local maximum or parameter convergence.

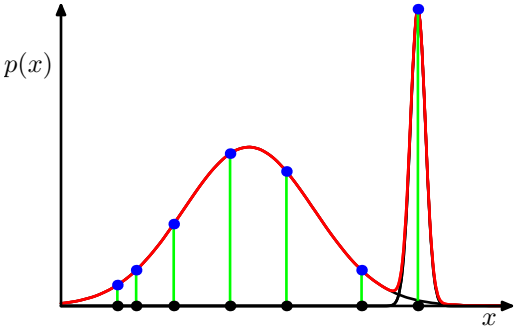
EM Convergence Properties: what the guarantee says

Exact E-steps and M-steps give $\ell(\boldsymbol{\theta}^{(t+1)}) \geq \ell(\boldsymbol{\theta}^{(t)})$.

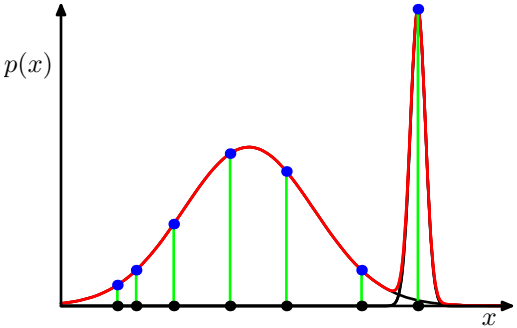
- If the likelihood values are bounded above, these values converge.
- This alone does not guarantee a local maximum or parameter convergence.
- General GMM likelihoods can be unbounded: a component can collapse onto one input.

Inspect covariances and effective counts as well as the likelihood.

Diagnosis: why an ever-rising likelihood can be a warning

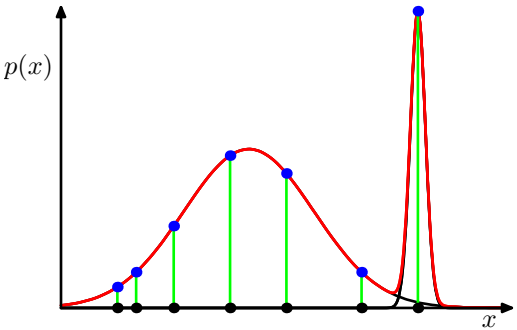


Diagnosis: why an ever-rising likelihood can be a warning



Place one mean at an observed input $\mathbf{x}_i$ and let its covariance shrink.

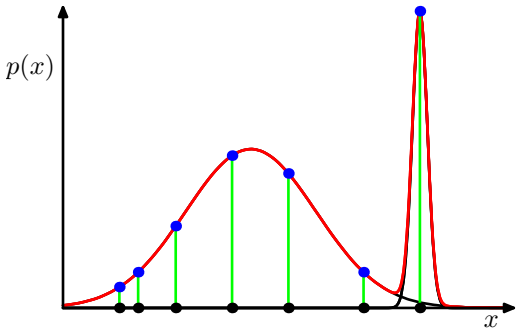
Diagnosis: why an ever-rising likelihood can be a warning



Place one mean at an observed input $\mathbf{x}_i$ and let its covariance shrink.

$$\mathcal{N}(\mathbf{x}_i \mid \mathbf{x}_i, \epsilon \mathbf{I}) = (2\pi\epsilon)^{-D/2} \to \infty.$$

Diagnosis: why an ever-rising likelihood can be a warning



Place one mean at an observed input $\mathbf{x}_i$ and let its covariance shrink.

$$\mathcal{N}(\mathbf{x}_i \mid \mathbf{x}_i, \epsilon\mathbf{I}) = (2\pi\epsilon)^{-D/2} \to \infty.$$

Other components can still explain the remaining inputs. A larger likelihood may mean collapse.

Use covariance constraints or regularization, and monitor tiny N_k.

Practical Considerations: connect each fix to a failure

- **Different final fits:** initialize with K-means and compare restarts.

Practical Considerations: connect each fix to a failure

- **Different final fits:** initialize with K-means and compare restarts.
- **Tiny covariance or effective count:** constrain covariance eigenvalues, regularize, or restart the component.

Practical Considerations: connect each fix to a failure

- **Different final fits:** initialize with K-means and compare restarts.
- **Tiny covariance or effective count:** constrain covariance eigenvalues, regularize, or restart the component.
- **Underflow in tiny densities:** normalize log weights using log-sum-exp.

Practical Considerations: connect each fix to a failure

- **Different final fits:** initialize with K-means and compare restarts.
- **Tiny covariance or effective count:** constrain covariance eigenvalues, regularize, or restart the component.
- **Underflow in tiny densities:** normalize log weights using log-sum-exp.
- **Choosing K:** compare held-out fit; AIC/BIC are useful heuristics, with care for mixture singularities.

Summary of EM for GMM: one complete iteration

E: freeze $\theta^{(t)}$. For every input, normalize component weights:

$$q_{kn}^{(t)} \propto \pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}), \quad \sum_k q_{kn}^{(t)} = 1.$$

Summary of EM for GMM: one complete iteration

E: freeze $\theta^{(t)}$. For every input, normalize component weights:

$$q_{kn}^{(t)} \propto \pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}), \quad \sum_k q_{kn}^{(t)} = 1.$$

M: freeze $q^{(t)}$. Set $N_k = \sum_n q_{kn}^{(t)}$ and refit:

$$\pi_k^{(t+1)} = N_k/N, \qquad \boldsymbol{\mu}_k^{(t+1)} = \sum_n q_{kn}^{(t)} \mathbf{x}_n / N_k.$$

Summary of EM for GMM: one complete iteration

E: freeze $\theta^{(t)}$. For every input, normalize component weights:

$$q_{kn}^{(t)} \propto \pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}), \quad \sum_k q_{kn}^{(t)} = 1.$$

M: freeze $q^{(t)}$. Set $N_k = \sum_n q_{kn}^{(t)}$ and refit:

$$\pi_k^{(t+1)} = N_k/N, \qquad \boldsymbol{\mu}_k^{(t+1)} = \sum_n q_{kn}^{(t)} \mathbf{x}_n / N_k.$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top / N_k.$$

Summary of EM for GMM: one complete iteration

E: freeze $\theta^{(t)}$. For every input, normalize component weights:

$$q_{kn}^{(t)} \propto \pi_k^{(t)} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}), \quad \sum_k q_{kn}^{(t)} = 1.$$

M: freeze $q^{(t)}$. Set $N_k = \sum_n q_{kn}^{(t)}$ and refit:

$$\pi_k^{(t+1)} = N_k/N, \qquad \boldsymbol{\mu}_k^{(t+1)} = \sum_n q_{kn}^{(t)} \mathbf{x}_n / N_k.$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \sum_n q_{kn}^{(t)} (\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_n - \boldsymbol{\mu}_k^{(t+1)})^\top / N_k.$$

Evaluate $\ell(\boldsymbol{\theta}^{(t+1)})$, check for degeneracy, then repeat.
Why it improves: the touching lower bound. Why it can fail: covariance collapse.

Applications of EM: identify what is missing

Model or setting	Hidden quantity
Gaussian mixture	Component label of each input

Applications of EM: identify what is missing

Model or setting	Hidden quantity
Gaussian mixture	Component label of each input
Hidden Markov model	Sequence of hidden states

Applications of EM: identify what is missing

Model or setting	Hidden quantity
Gaussian mixture	Component label of each input
Hidden Markov model	Sequence of hidden states
Missing-data model	Unobserved entries

Applications of EM: identify what is missing

Model or setting	Hidden quantity
Gaussian mixture	Component label of each input
Hidden Markov model	Sequence of hidden states
Missing-data model	Unobserved entries

E: infer a distribution over what is missing.
M: maximize the expected complete-data log-likelihood.

Optional extension: EM and Variational Inference

The same identity guides both methods:

$$\ell(\boldsymbol{\theta}) = \mathcal{F}(q, \boldsymbol{\theta}) + D_{\mathrm{KL}}(q \| p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})).$$

Optional extension: EM and Variational Inference

The same identity guides both methods:

$$\ell(\boldsymbol{\theta}) = \mathcal{F}(q, \boldsymbol{\theta}) + D_{\mathrm{KL}}(q \| p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})).$$

- **Standard EM:** compute the exact posterior in the E-step.
  For GMMs, this is the responsibility formula.

Optional extension: EM and Variational Inference

The same identity guides both methods:

$$\ell(\boldsymbol{\theta}) = \mathcal{F}(q, \boldsymbol{\theta}) + D_{\mathrm{KL}}(q \| p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})).$$

- **Standard EM:** compute the exact posterior in the E-step.
  For GMMs, this is the responsibility formula.
- **Variational EM:** restrict q to a tractable family.
  Improve the ELBO over this family, then update $\boldsymbol{\theta}$.

Optional extension: EM and Variational Inference

The same identity guides both methods:

$$\ell(\boldsymbol{\theta}) = \mathcal{F}(q, \boldsymbol{\theta}) + D_{\mathrm{KL}}(q \| p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta})).$$

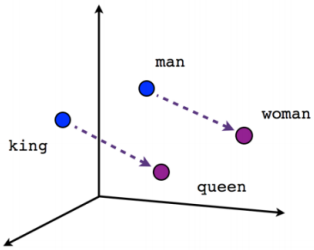
- **Standard EM:** compute the exact posterior in the E-step.
  For GMMs, this is the responsibility formula.
- **Variational EM:** restrict q to a tractable family.
  Improve the ELBO over this family, then update $\boldsymbol{\theta}$.

A restricted family may leave a positive gap; improving its ELBO need not improve the true likelihood.

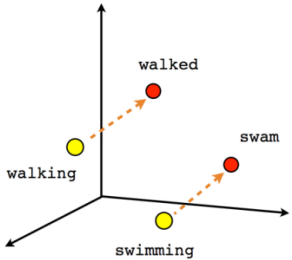
Table of Contents

3 Optional: more unsupervised examples

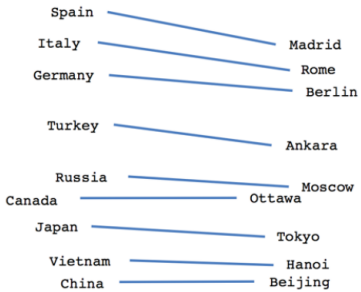
Learning word representations using `word2vec` (Mikolov et al. 2013).



Male-Female



Verb tense



Country-Capital

Given voting patterns of regions across Switzerland, we use PCA to extract useful features (Etter et al. 2014).

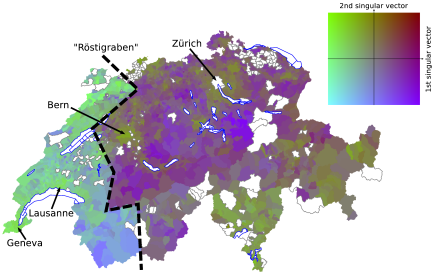
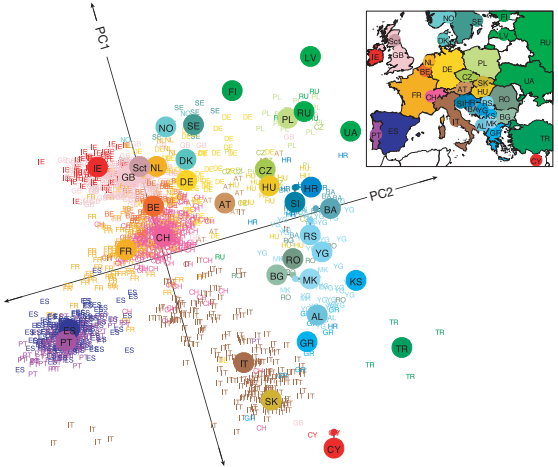


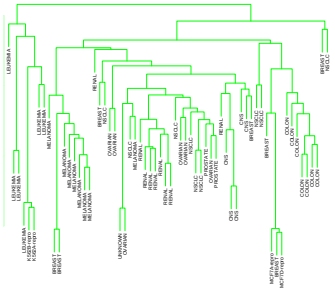
Figure 9: Voting patterns of Swiss municipalities. The color of a municipality is assigned using its location in Figure 8 and the color gradient shown in the upper right corner. Two municipalities with similar colors have similar voting patterns. The *Röstigraben*, corresponding to the cultural difference between French-speaking municipalities and German-speaking ones, is clearly visible from the difference in voting patterns. Regions shown in white are lakes or municipalities for which some vote results are missing (due to a merging of municipalities, for example). A more detailed map can be found online [2].

PCA Example 2: Genes mirror geography



Nature 2008, `http://dx.doi.org/10.1038/nature07331`

Examples for Clustering


LEUKEMIA
LEUKEMIA
LEUKEMIA
LEUKEMIA
LEUKEMIA
LEUKEMIA
LEUKEMIA
MELANOMA
MELANOMA
MELANOMA
MELANOMA
MELANOMA
MELANOMA
MELANOMA
BREAST
BREAST
BREAST
BREAST
BREAST
RENAL
RENAL
RENAL

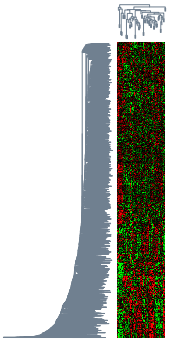


FIGURE 14.14. *DNA microarray data: average linkage hierarchical clustering has been applied independently to the rows (genes) and columns (samples), de-*

Clustering articles published in Science (Blei & Lafferty 2007)

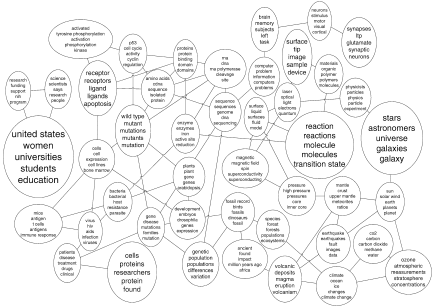


FIG. 2. *A portion of the topic graph learned from* 16,351 *OCR articles from Science* (1990–1999). *Each topic node is labeled with its five most probable phrases and has font proportional to its popularity in the corpus. (Phrases are found by permutation test.) The full model can be found in* http://www.cs.cmu.edu/~lemur/science/ *and on STATLIB.*

How does it work?

Define an unsupervised or self-supervised loss function, for

How does it work?

Define an unsupervised or self-supervised loss function, for

- Compression & Reconstruction (e.g. Auto-Encoder)

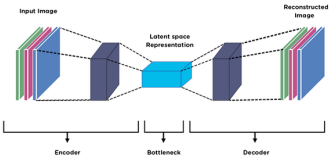


image: Deepak Birla

How does it work?

Define an unsupervised or self-supervised loss function, for

- Compression & Reconstruction (e.g. Auto-Encoder)

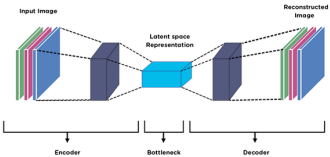


image: Deepak Birla

- Consistency & Contrastive Learning (e.g. Noise-contrastive estimation)

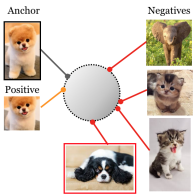


image: arxiv.org/pdf/2004.11362

Representation and generation are different goals

- **Auto-encoder:** reconstruct an input through a compressed representation.

Representation and generation are different goals

- **Auto-encoder:** reconstruct an input through a compressed representation.
- **Generative model:** specify how to sample a new input.
  A GMM samples a component, then samples from its Gaussian.

Representation and generation are different goals

- **Auto-encoder:** reconstruct an input through a compressed representation.
- **Generative model:** specify how to sample a new input.
  A GMM samples a component, then samples from its Gaussian.
- A plain auto-encoder alone does not specify a distribution to sample in its latent space.

Representation and generation are different goals

- **Auto-encoder:** reconstruct an input through a compressed representation.
- **Generative model:** specify how to sample a new input.
  A GMM samples a component, then samples from its Gaussian.
- A plain auto-encoder alone does not specify a distribution to sample in its latent space.
- Language models can learn from prediction targets constructed from text itself.

- Generative Adversarial Networks (GAN) (G)
  see also *predictability minimization*



“A Style-Based Generator Architecture for Generative Adversarial Networks”, CVPR 2019

- Contrastive image-language pretraining (CLIP)
  learns a joint representation space for images and text using contrastive learning

- Diffusion models (G)
  learn to reverse a noising process; some models condition generation on text prompts



Source: Stable Diffusion model, `https://stability.ai/blog/stable-diffusion-public-release`